Defensa completa del proyecto

Zombis vs. Plantas · IPC1 · Proyecto 2 · 2S2026

Recorridos del programa, fichas de las 42 clases, métodos declarados, preguntas, ejemplos y correspondencia con la rúbrica.

Alcance comprobado: 42 archivos Java de la aplicación; 332 nombres de métodos, constructores y callbacks declarados documentados. Sobrecargas y callbacks del mismo nombre se agrupan. Los métodos heredados del JDK no se catalogan; sí se explican los utilizados en los recorridos.

Cómo utilizar la guía

• **Páginas 4–20:** lectura inicial del flujo completo y fragmentos explicados. Es la parte para preparar una exposición.

• **Desde la página 21:** consulta de las 42 clases. Cada ficha explica responsabilidad, estado, recorrido, fragmento, métodos y pregunta.

• **Índices en páginas 2–3:** localiza una clase. El PDF también tiene marcadores de navegación por paquete y clase.

• **Al final:** recorridos con valores, decisiones reales del código y evidencia que corresponde a los 100 puntos de la hoja.

> “Para explicar una función: identifica qué recibe, qué condición comprueba, qué atributo cambia, qué devuelve y qué se ve en pantalla. Sigue una acción entre clases y practica con tus propias palabras.”

Archivos a tener abiertos al presentar

NetBeans con el proyecto, los dos manuales PDF, Niveles y la carpeta de cuentas. Inicia por main; muestra la aplicación y alterna al método relacionado. Usa Ctrl + F para buscar nombres. Los paquetes permanecen alfabéticos; las pestañas se arrastran manualmente.

Las fichas corresponden a la versión corregida del repositorio. Si cambias el código, los números de línea pueden variar. La guía describe lo que hace la implementación, incluidos sus límites, en lugar de atribuirle comportamientos que no realiza.

Índice de clases · 1/2

Paquete	Clase	Página
Principal	Proyecto2_PlantasVsZombies.java	21
Principal	Juego.java	22
Archivos	RutasCuentas.java	23
Archivos	GestorCuentas.java	24
Archivos	LectorNivel.java	26
Archivos	GestorPartidas.java	28
Modelo	Cuenta.java	29
Modelo	Estadisticas.java	30
Modelo	ConfiguracionNivel.java	32
Modelo	Casilla.java	33
Modelo	Jardin.java	34
Modelo	Partida.java	35
Modelo	Proyectil.java	37
Modelo	Cerebro.java	38
Modelo	MotorCombate.java	39

Recorridos y respuestas al final

Recorridos completos: página 74.

Ejercicios con valores: página 75.

Decisiones y límites: página 76.

Evidencia de la rúbrica: página 77.

Fundamentos de Java: página 78.

Archivos y diagnóstico: página 79.

Índice de clases · 2/2

Paquete	Clase	Página
Plantas	Planta.java	42
Plantas	Lanzaguisantes.java	43
Plantas	Patatapum.java	44
Plantas	Hielaguisantes.java	45
Plantas	Nuez.java	46
Plantas	Carronivora.java	47
Plantas	Pinchohierba.java	48
Plantas	Bipetidora.java	49
Zombis	Zombi.java	50
Zombis	ZombiComun.java	52
Zombis	ZombiCaracono.java	53
Zombis	ZombiGarrocha.java	54
Zombis	ZombiDeportista.java	55
Zombis	ZombiBailon.java	56
Zombis	ZombiBailarin.java	57
Zombis	Zombistein.java	58
Zombis	Zombidito.java	59
Vista	Recursos.java	60
Vista	PanelFondo.java	61
Vista	PanelJardinFondo.java	62
Vista	HilosPartida.java	64
Vista	VentanaInicio.java	65
Vista	VentanaCrearCuenta.java	66
Vista	VentanaSeleccionarCuenta.java	68
Vista	VentanaMenuPrincipal.java	70
Vista	VentanaNuevaPartida.java	71
Vista	VentanaPartida.java	72

1. Cómo arranca y se conecta el programa

Relación con la rúbrica: Main de arranque, distribución de responsabilidades y navegación.

Fragmento real: Proyecto2_PlantasVsZombies.java · líneas 5–8.

```
public static void main(String[] args) {

    javax.swing.SwingUtilities.invokeLater(() -> new Juego().iniciar());
}
```

• **main(String[] args):** es el punto por el que Java inicia la aplicación. args recibe posibles argumentos de ejecución.

• **invokeLater(...):** coloca la tarea de apertura en la cola del EDT, el hilo que atiende eventos y componentes de Swing.

• **() -> ...:** es una lambda: describe una tarea sin ejecutarla inmediatamente en esa misma expresión.

• **new Juego().iniciar():** construye el coordinador de inicio y solicita abrir la primera ventana.

Qué hace Juego

Su constructor crea GestorCuentas, que carga las cuentas existentes. iniciar() construye VentanaInicio con ese mismo gestor, centra la ventana y la hace visible. El main no contiene las reglas de combate.

Recorrido de llamadas que debes poder seguir

Acción	Clases que intervienen
Abrir la aplicación	main → Juego → GestorCuentas → VentanaInicio.
Crear o seleccionar cuenta	VentanaCrearCuenta / VentanaSeleccionarCuenta → GestorCuentas → Cuenta → VentanaMenuPrincipal.
Iniciar nivel	VentanaNuevaPartida → LectorNivel → ConfiguracionNivel → Jardin → Partida → MotorCombate → VentanaPartida.
Jugar	HilosPartida publica tareas → fotograma() → motor.actualizar(dt) → actualizar etiquetas y repaint().
Guardar y recuperar	VentanaPartida → GestorPartidas → archivo binario → reconstrucción de la ventana pausada.

> “El programa separa la presentación, los datos y las operaciones de archivos. La ventana recibe acciones; el modelo conserva el estado y aplica reglas; los gestores realizan la lectura y escritura.”

Pregunta de práctica

¿Por qué pasar GestorCuentas y Cuenta entre ventanas? Para mantener la cuenta activa y utilizar la información cargada, en lugar de perder el contexto al navegar. Crear una ventana no implica empezar otra partida.

2. Cuentas: validar, buscar y escribir

Relación con la rúbrica: Creación, unicidad sensible a mayúsculas, selección y archivos por cuenta.

Fragmento real: GestorCuentas.java · líneas 23–43.

```
public boolean nicknameValido(String nickname) {
    if (nickname == null || nickname.length() < 1 || nickname.length() > 12) {
        return false;
    }

    for (int i = 0; i < nickname.length(); i++) {
        if (!Character.isLetterOrDigit(nickname.charAt(i))) {
            return false;
        }
    }
    return true;
}

public Cuenta buscarCuenta(String nickname) {
    for (int i = 0; i < cantidadCuentas; i++) {
        if (cuentas[i].getNickname().equals(nickname)) {
            return cuentas[i];
        }
    }
    return null;
}
```

• **Primera condición:** rechaza null y longitudes fuera de 1–12. El cortocircuito de || evita consultar length() si la referencia es null.

• **for:** recorre cada carácter; isLetterOrDigit() admite letras o dígitos. Un carácter inválido devuelve false inmediatamente.

• **equals():** compara el contenido exacto del nickname. Ana1 y ana1 son diferentes.

• **Retornos:** buscarCuenta() devuelve el objeto encontrado o null. nicknameValido() devuelve true únicamente cuando pasa todas las comprobaciones.

Secuencia al pulsar Crear cuenta

El evento de VentanaCrearCuenta toma el texto y aplica trim(), que elimina espacios de los extremos. crearCuenta() valida, comprueba que no exista, prepara espacio en el arreglo y crea la carpeta, partidas y estadisticas.txt. Después agrega un objeto Cuenta y aumenta cantidadCuentas. Si falla la preparación de archivos devuelve false.

Rutas y recuperación

RutasCuentas: genera u- y cuatro dígitos hexadecimales por carácter. Ana se codifica u-0041006e0061; ana, u-0061006e0061. Esto evita depender de si el disco distingue mayúsculas. nickname() realiza la conversión inversa.

cargarCuentas(): enumera carpetas, recupera el nickname, carga las ocho líneas del archivo de estadísticas y crea Cuenta. El arreglo inicial puede ampliarse; solo cantidadCuentas posiciones contienen cuentas registradas.

> “La unicidad se comprueba sobre el texto exacto y la ruta conserva esa distinción. Cuenta relaciona el nickname con un objeto Estadisticas; GestorCuentas administra su persistencia.”

3. JSON: cómo funciona el analizador propio

Relación con la rúbrica: Lectura java.io, validación del formato y rechazo de niveles inválidos.

Entrada, proceso y salida

Entrada: un File elegido en JFileChooser. leerNivel() lee UTF-8 en bloques de caracteres, limita el tamaño y entrega el texto a Analizador. **Salida válida:** ConfiguracionNivel con duración, cerebros iniciales y String[][] del jardín. Una excepción se muestra como mensaje de nivel inválido.

Fragmento real: LectorNivel.java · líneas 19–30.

```
private static class Analizador {
    private final String texto; private int posicion;
    Analizador(String texto) { this.texto = texto; }
    private void espacios() {
        while (posicion < texto.length() && Character.isWhitespace(texto.charAt(posicion)))
            posicion++;
    }
    private boolean tomar(char c) {
        espacios();
        if (posicion < texto.length() && texto.charAt(posicion) == c) { posicion++; return true;
            }
        return false;
    }
    private void exigir(char c) { if (!tomar(c)) throw error("Se esperaba '" + c + "'"); }
```

• **texto y posicion:** conservan el contenido y el carácter que toca interpretar.

• **espacios():** avanza sobre espacios en blanco para que la presentación del JSON no afecte los separadores.

• **tomar(c):** consume el símbolo solo si coincide y devuelve si lo consiguió. Permite comprobar comas opcionales en los recorridos.

• **exigir(c):** requiere ese símbolo; si falta, genera una excepción con la posición del problema.

El resto del análisis

cadena(): exige comillas, recorre hasta el cierre y devuelve el texto. **entero():** recorre los dígitos y usa Integer.parseInt(). **clave():** comprueba el nombre esperado y los dos puntos.

leer(): exige las tres claves en el orden del formato; valida duración positiva y cerebros no negativos; recorre filas y plantas con do/while; compara el ancho de cada fila y exige el cierre completo sin contenido adicional.

Los arreglos temporales crecen cuando se llenan y al finalizar se copian a arreglos del tamaño exacto. valida() admite VACIO y los siete identificadores de plantas. No es un lector universal de cualquier variante de JSON: implementa el esquema del proyecto.

> “Si una fila tiene tres columnas y la siguiente cuatro, el analizador detecta la diferencia antes de crear Jardin. Así la construcción recibe una matriz rectangular válida.”

4. Jardín, casillas y estado del modelo

Relación con la rúbrica: Construcción dinámica, composición y organización de datos.

Fragmento real: Jardin.java · líneas 19–27.

```
public Jardin(String[][] configuracion) {

    filas = configuracion.length;
    columnas = configuracion[0].length;

    casillas = new Casilla[filas][columnas];

    crearJardin(configuracion);
}
```

Qué significa cada dato

• **configuracion.length:** cantidad de filas. configuracion[0].length es el ancho de la primera fila, ya validado por el lector.

• **new Casilla[filas][columnas]:** reserva la matriz de referencias; los objetos Casilla se crean dentro de crearJardin().

• **Dos for anidados:** el exterior cambia de fila y el interior recorre sus columnas. Para cada celda se crea una Casilla y se llama a crearPlanta().

• **VACIO:** crearPlanta() devuelve null. Los demás identificadores reconocidos producen el objeto de la planta concreta, que se coloca en la casilla.

Clase	Estado que representa
ConfiguracionNivel	Datos iniciales leídos del JSON. Sus dimensiones se obtienen de String[][].
Casilla	Fila, columna y referencia Planta. estaVacia() compara con null; quitarPlanta() retira esa referencia.
Jardin	Matriz Casilla[][], dimensiones, creación y siembra en espacios disponibles.
Partida	Cuenta, jardín, dificultad, reloj, cerebros, zombis, compras, gastos, podadoras y banderas de pausa/final.
MotorCombate	Coordina reglas y conserva proyectiles, cerebros visibles y plazos internos de los efectos.

Ejemplo de composición

> “La partida referencia un jardín; el jardín contiene casillas y una casilla puede contener una planta. Quitar la planta cambia el contenido de la casilla, no elimina la casilla ni cambia el tamaño del tablero.”

Pregunta: ¿por qué separar Partida y MotorCombate? Partida conserva el estado y operaciones de control; el motor coordina su evolución. La vista consulta ambos para mostrar los datos.

5. Del clic a la compra de un zombi

Relación con la rúbrica: Selección de tipo y carril, costo, validaciones y contadores.

1) Un botón del catálogo guarda el índice del tipo seleccionado. 2) PanelJardinFondo traduce la posición vertical del clic a fila. 3) La interfaz SeleccionCarril notifica a VentanaPartida.invocar(fila). 4) crear() construye el tipo concreto y se llama a Partida.invocarZombi().

Fragmento real: Partida.java · líneas 143–158.

```
public boolean invocarZombi(Zombi zombi) {
    if (zombi == null || pausada || terminada
            || zombi.getCosto() <= 0
            || zombi.getFila() < 0
            || zombi.getFila() >= jardin.getFilas()
            || costoActual(zombi.getCosto()) > cerebros) {
        return false;
    }

    guardarEnArreglo(zombi);
    int costo = costoActual(zombi.getCosto());
    cerebros -= costo;
    cerebrosGastados += costo;
    zombisComprados++;
    return true;
}
```

• **Condición inicial:** rechaza referencias nulas, pausa, partida terminada, costo no comprable, fila inexistente y falta de cerebros. Devuelve false sin modificar el estado.

• **guardarEnArreglo():** guarda la referencia del zombi. Si el arreglo está lleno, crea uno de doble capacidad y copia las posiciones existentes.

• **costoActual():** calcula el precio según el bloque de horda. Ese mismo valor se descuenta y se acumula como gasto.

• **zombisComprados++:** cuenta una compra exitosa. El true permite a la ventana actualizar la selección y el tablero.

Ejemplo con valores

Con 150 cerebros, sin descuento, comprar Caracono cuesta 100: el saldo queda en 50, cerebrosGastados sube 100 y zombisComprados sube 1. Intentar comprar después un Deportista de 150 devuelve false y mantiene esos valores.

> “El clic inicia la solicitud, pero la validación final está en Partida. La ventana informa el error si la compra no se puede realizar. agregarZombiGenerado() registra bailarines o Zombidito sin cobrar ni aumentar las compras.”

6. Reloj y actualización del motor

Relación con la rúbrica: Temporizador visible, movimiento continuo y coordinación de procesos.

VentanaPartida.fotograma()

System.nanoTime() mide el intervalo desde la última actualización. Se convierte de nanosegundos a segundos y se limita a 0.15. fraccion acumula pasos pequeños; por cada segundo completo se descuenta uno del reloj. El mismo dt avanza el motor.

> “Si fraccion vale 0.97 y llega un paso de 0.06, pasa a 1.03: se descuenta un segundo y quedan 0.03 para el siguiente ciclo. Por eso no se pierde el tiempo fraccionario entre fotogramas.”

Fragmento real: MotorCombate.java · líneas 96–113.

```
public void actualizar(double segundos) {
    if (segundos <= 0 || partida.estaPausada() || partida.estaTerminada()) return;
    segundos = Math.min(segundos, 0.15);
    ahora += segundos;
    actualizarEconomia();
    actualizarDefensa();
    for (int f = 0; f < animacionPodadoras.length; f++) {
        if (animacionPodadoras[f] > 0) {
            animacionPodadoras[f] = Math.max(0, animacionPodadoras[f] - segundos);
        }
    }
    asegurarEspacioZombis();
    activarPatatapumsYDigestion();
    dispararPlantas();
    moverProyectiles(segundos);
    moverYAtacarZombis(segundos);
    pinchohierbaYPatatapum();
}
```

• **Retorno inicial:** sin tiempo positivo o con pausa/final, no hay actualización.

• **ahora += segundos:** avanza el tiempo interno del motor. Los vencimientos se comparan con ese tiempo de juego.

• **Economía y defensa:** primero vencen cerebros y se evalúa si corresponde sembrar.

• **Estados, disparos y combate:** se preparan plantas, crean y mueven proyectiles, avanzan y atacan zombis y se resuelven reglas de casilla.

Después de actualizar, la ventana cambia etiquetas y solicita repaint(). Si el motor comunica victoria llama a finalizar(true); si el reloj llegó a cero llama a finalizar(false). El límite de dt evita un salto grande de simulación ante una demora, pero una demora prolongada no se recupera completamente como tiempo real.

7. Plantas: datos, herencia y plazos

Relación con la rúbrica: Las siete plantas, su daño, cadencia y estados gráficos.

Fragmento real: Lanzaguisantes.java · líneas 8–14.

```
public Lanzaguisantes(int fila, int columna) {

    super(100, fila, columna);

    danio = 20;
    cadenciaDisparo = 1000;
}
```

super(100, fila, columna): inicializa la parte heredada de Planta. danio y cadenciaDisparo pertenecen a Lanzaguisantes. La cadencia está en milisegundos; el motor la convierte a segundos al programar el siguiente disparo.

Clase	Cómo se ejecuta su comportamiento
Planta	Clase abstracta con salud y ubicación. recibirDanio() resta y limita la salud a cero; estaViva() comprueba si es positiva.
Lanzaguisantes	dispararPlantas() busca un blanco en dirección frontal, crea un Proyectil de daño 20 y programa el siguiente disparo a 1 segundo.
Hielaguisantes	Crea proyectiles de hielo con daño 15 cada 1.5 s. El impacto escribe congeladoHasta = ahora + 4; el avance se multiplica por 0.60 mientras ese plazo esté activo.
Patatapum	preparadaEn guarda cuándo se cumplen los 10 s. Entonces activar() cambia su bandera; al coincidir zombis en la casilla aplica 100 de daño y se retira.
Nuez	Tiene 400 de salud. obtenerEstadoVisual() devuelve 1, 2 o 3 por porcentaje; el panel representa grietas y daño. El motor impide que Garrocha la supere.
Carronivora	Si no está digiriendo, revisa su casilla y la frontal. Engulle o aplica 200 a Zombistein. digestionHasta marca ahora + 8; al vencer terminaDigestion().
Pinchohierba	No recibe mordiscos normales. proximoPincho programa 10 de daño cada 0.5 s en su casilla. Zombistein la retira mediante destruirPlanta().
Bipetidora	Busca blancos por separado a ambos lados. Crea un proyectil frontal y dos traseros de 15 de daño; utiliza una separación visual pequeña para la ráfaga.

> “Los estados temporales se guardan como plazos del reloj de juego. No se duerme durante ocho segundos dentro del ataque de Carroñívora; el motor sigue trabajando y comprueba cuándo termina su digestión.”

8. Zombis: estado particular y polimorfismo

Relación con la rúbrica: Catálogo, escudos, salto, baile, aplastamiento y lanzamiento.

La clase base Zombi

Conserva salud actual y máxima, daño, costo, modificador de velocidad, fila, posición y atacando. mover() solo cambia posicionX si está vivo y no ataca. perdioBrazo() compara salud con la mitad del máximo; el panel selecciona la imagen.

Clase concreta	Qué añade o cambia
ZombiComun	Inicializa el zombi base: 100 HP, daño 15, costo 50 y velocidad ×1.
ZombiCaracono	Escudo 100. Redefine recibirDanio() para consumir primero el escudo y getDanio() para usar 20 con cono o 15 sin él.
ZombiDeportista	Escudo 150 y velocidad ×2. Redefine el daño recibido; su ataque de 10 se programa cada 0.5 s.
ZombiGarrocha	tieneGarrocha y saltoRealizado impiden usos repetidos. realizarSalto() consume la garrocha y cambia la velocidad de ×1.7 a ×1.
ZombiBailon	Bandera de baile e intervalos. El motor detiene su avance durante 1 s e invoca adyacentes cada 5 s cuando hay espacio.
ZombiBailarin	Entidad secundaria creada por el Bailón; no pertenece al catálogo de compra.
Zombistein	Aplasta plantas. debeLanzarZombidito() exige salud a la mitad o menos y que todavía no haya lanzado. La bandera evita repetirlo.
Zombidito	Entidad secundaria de 50 HP y velocidad ×1.7; el motor conserva origen y plazo de su vuelo.

Ejemplo exacto del escudo

En recibirDanio(), si cantidad es menor o igual al escudo se resta solo del escudo. Si lo supera, se calcula danioRestante = cantidad − escudo, se pone el escudo en cero y se llama a super.recibirDanio(danioRestante).

> “Con 30 de escudo y un golpe de 50, quedan cero de escudo y se pierden 20 de salud. El motor utiliza una referencia Zombi, pero Java ejecuta el recibirDanio() del tipo real: ese es el polimorfismo.”

Detalle: ante Nuez el motor consume la garrocha y reduce la velocidad, pero no traslada al saltador al otro lado. Zombistein usa aplastamiento, no mordiscos; su daño base cero no significa que sea inofensivo.

9. Movimiento, colisiones y casilla lógica

Relación con la rúbrica: Movimiento continuo, primer impacto, ataques y reglas espaciales.

Mover y frenar delante de una planta

moverYAtacarZombis() recorre los zombis vivos. Busca la planta bloqueante más cercana por delante en el mismo carril; omite Pinchohierba. Calcula un avance base de 0.44 columnas/s × dt, aplica hielo y limita el movimiento para no atravesar el obstáculo. Zombi.mover() aplica además el modificador de velocidad de su clase.

> “Con dt = 0.10, el avance base es 0.044 columnas. Un Deportista lo multiplica por 2 y avanza 0.088 si no hay obstáculos. Con hielo se multiplica además por 0.60: avanza 0.0528.”

Impactos sin saltarse blancos

moverProyectiles() guarda la posición anterior, llama avanzar(dt) y obtiene el tramo recorrido. Revisa zombis vivos del mismo carril cuyo centro esté en el tramo. Conserva el de menor distancia al origen del paso, aplica daño y retira el proyectil. Los zombis en vuelo se excluyen de estos blancos.

Fragmento real: Proyectil.java · líneas 31–31.

```
public void avanzar(double segundos) { x += direccion * velocidad * segundos; }
```

direccion vale −1 o 1. velocidad vale 3 columnas/s. El modelo trabaja en columnas; el panel transforma esas medidas a píxeles. Esto conserva la lógica al cambiar el tamaño de la ventana.

Casilla única para efectos locales

Fragmento real: MotorCombate.java · líneas 473–476.

```
public int casillaActual(Zombi z) {
    int c = (int) Math.floor(z.getPosicionX() + 0.35);
    return Math.max(0, Math.min(jardin.getColumnas() - 1, c));
}
```

floor() obtiene el entero inferior de la posición del centro. Los límites evitan índices fuera de la matriz. Si posicionX es 1.8, el centro es 2.15 y la casilla es 2, usando índices desde cero.

Ataques y eliminación

atacarConMordisco() comprueba proximoMordisco: el común espera 1 s y Deportista 0.5 s. Una planta sin salud se retira y genera cerebros. Los zombis muertos permanecen como referencias en el arreglo de la partida, pero las reglas y el pintado los omiten; los proyectiles usados sí se retiran de su arreglo activo.

10. Cerebros: generación, vigencia y clic

Relación con la rúbrica: Generadores según dificultad, plazos aleatorios y recolección.

Cómo se representa un cerebro

Cerebro guarda x, y, vence y generador. x/y están en columnas y carriles; vence es un instante del reloj de juego. generador indica qué generador lo produjo; −1 identifica uno liberado por una planta destruida.

Fragmento real: MotorCombate.java · líneas 499–499.

```
private double esperaCerebro() { return 5 + azar.nextInt(11); }
```

nextInt(11) produce un entero entre 0 y 10; al sumar 5, el plazo queda entre 5 y 15 s, incluidos. El constructor crea proximoCerebro con 6, 5 o 4 posiciones según la dificultad e inicializa cada vencimiento por separado.

El ciclo de cada generador

• actualizarGenerador(indice) comprueba que la partida permita jugar y que haya vencido su plazo.

• Crea un Cerebro en una posición aleatoria y con vence = ahora + 5.

• Escribe Double.POSITIVE_INFINITY como próxima emisión mientras ese icono siga presente, evitando duplicarlo en cada actualización.

• Al recogerlo o vencer, quitarCerebro() programa una nueva espera aleatoria para ese generador.

Qué hace el clic

PanelJardinFondo revisa primero los iconos, del último al primero, para atender el que se dibuja encima. Si el clic cae en un cerebro, llama recogerCerebro(), solicita repaint() y retorna para no comprar también un zombi con el mismo clic.

recogerCerebro() rechaza pausa, final e índices inválidos. Si el icono ya venció, lo retira sin entregar recursos; si sigue vigente, lo retira y agrega 25 cerebros a Partida.

> “Con saldo 50, pulsar un cerebro vigente deja 75. Pulsarlo después de su vencimiento no agrega nada. La duración se mide en tiempo de juego, por eso la pausa también conserva su vigencia restante.”

Planta destruida

destruirPlanta() calcula 3 + nextInt(4): produce de 3 a 6 iconos. Cada uno vence a los 5 s y usa generador −1; retirarlo no reprograma ninguno de los generadores periódicos.

11. IA: cómo elige una única defensa

Relación con la rúbrica: Fácil, Medio y prioridades de Difícil.

Control del momento de actuar

actualizarDefensa() retorna si la dificultad es Fácil o todavía no llegó proximaDefensa. Si corresponde evaluar, programa el siguiente turno: 10 s en Medio o 5 s en Difícil. Esto evita sembrar en cada fotograma.

Medio

Elige uno de los siete tipos al azar. Recorre columnas desde la última hasta la primera y, dentro de cada columna, las filas. La primera siembra exitosa termina el método. Así busca desde el lado de la casa, que está a la derecha.

Difícil: regla exterior, zombis interiores

Un for recorre reglas 0–5 y otro revisa todos los zombis vivos. Esta estructura hace que se examine primero una prioridad en todo el tablero antes de considerar la siguiente. Si una acción consigue sembrar, return finaliza el turno.

Prioridad	Condición y acción del código
1. Cerca de la casa	Si quedan como máximo 4 columnas hasta la última, intenta Nuez o Carroñívora por delante.
2. Rebasó una planta	Si hay una planta detrás del zombi, intenta Bipetidora en un espacio detrás.
3. Resistente	Si es Caracono o Zombistein, intenta Patatapum o Carroñívora por delante.
4. Deportista	Intenta Hielaguisantes por delante.
5. Garrocha	Intenta Nuez por delante.
6. Más de 3 en carril	Cuenta zombis vivos del carril e intenta Pinchohierba.
7. Alternativa final	Cuenta defensas por fila y busca la menos defendida con espacio para Lanzaguisantes.

> “sembrarEnCarril() empieza en la casilla siguiente al zombi y busca espacios hacia la casa. sembrar() comprueba que la casilla esté libre e inicializa plazos de la planta nueva; por ejemplo, una Patatapum recién sembrada debe esperar sus propios diez segundos.”

Pregunta: ¿es una IA que aprende? No: utiliza selección aleatoria en Medio y reglas de prioridad en Difícil. Su comportamiento se puede explicar siguiendo condiciones y recorridos.

12. Bloques, descuentos, podadoras y final

Relación con la rúbrica: Siete periodos, hordas, podadoras de un uso y resumen.

Fragmento real: Partida.java · líneas 194–202.

```
public int getBloque() {
    double transcurrido = duracionTotal - tiempoRestante + fraccionReloj;
    return Math.min(7, 1 + (int)(transcurrido * 7.0 / duracionTotal));
}
// El enunciado no fija redondeo para costos fraccionarios: se redondea arriba.
public int costoActual(int base) {
    int bloque = getBloque();
    return bloque == 7 ? (base + 3) / 4 : bloque == 4 ? (base + 1) / 2 : base;
}
```

• **transcurrido:** duración inicial menos tiempo restante, más la fracción todavía no descontada.

• **× 7 / duración:** transforma el tiempo a una posición entre los siete bloques. Se suma 1 porque el primero se numera 1; min limita al bloque 7.

• **Precio:** el operador ternario elige cuarto de precio en bloque 7, mitad en bloque 4 o precio base. Con enteros, (base + 3) / 4 redondea hacia arriba.

Ejemplo con 70 s: de 0 a menos de 10 s es bloque 1; a los 30 s empieza el 4; a los 40 vuelve el precio normal en el 5; a los 60 empieza el 7. El común pasa de 50 a 25, vuelve a 50 y finalmente cuesta 13.

Llegada a la casa

llegadaALaCasa(fila) consulta el booleano de podadora del carril. Si está disponible, activarPodadora() lo cambia a false, suma el contador y el motor aplica daño suficiente a todos los zombis vivos de esa fila. La animación dura 0.9 s. Si ya se usó, victoria pasa a true y termina la partida.

Registro del resultado

VentanaPartida.finalizar() comprueba resultadoRegistrado antes de actuar. Luego marca el final, detiene hilos, actualiza estadísticas una vez, intenta escribirlas y muestra el resumen. La derrota se decide cuando el reloj alcanza cero sin invasión.

> “La primera llegada no gana: consume la defensa del carril. Para ganar hay que lograr otra llegada por un carril cuya podadora ya se haya usado.”

Demostración corta

En un JSON separado usa 70 s, 500 cerebros y una fila de cuatro VACIO. En Fácil, sin comprar, puedes mostrar los descuentos y la derrota. En otra partida, un común consume la podadora y otro posterior permite demostrar victoria. No cambies los niveles originales para esta prueba.

13. Hilos: trabajadores y EDT de Swing

Relación con la rúbrica: Concurrencia y restricciones técnicas; continuidad al pausar.

Fragmento real: HilosPartida.java · líneas 39–49.

```
private Thread trabajador(String nombre, int actual, Runnable tarea) {
    // La validacion se ejecuta en el EDT, no en el hilo trabajador.
    Runnable entrega = () -> { if (activo && sesion == actual) tarea.run(); };
    Thread hilo = new Thread(() -> {
        while (!Thread.currentThread().isInterrupted()) {
            try { Thread.sleep(40); }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); return; }
            SwingUtilities.invokeLater(entrega);
        }
    }, nombre);
    hilo.setDaemon(true); hilo.start(); return hilo;
```

• **Runnable:** representa una tarea. entrega comprueba activo y el número de sesión antes de ejecutar tarea.run(). Esa comprobación ocurre en el EDT.

• **new Thread(...):** crea un trabajador real. El while continúa hasta una interrupción; sleep(40) evita que espere consumiendo CPU continuamente.

• **InterruptedException:** se restablece la marca de interrupción y se retorna para terminar el trabajador.

• **invokeLater(entrega):** publica el trabajo en la cola del EDT. El trabajador no modifica directamente el modelo ni los componentes.

• **daemon y start():** se marca como hilo auxiliar y se pone en ejecución; no se invoca run() directamente para arrancarlo.

Inicio y pausa

iniciar() exige EDT, evita un inicio duplicado, activa una sesión nueva y crea un reloj y 6/5/4 trabajadores generadores. Cada generador captura un índice final para actualizar su propio plazo.

detener() pone activo = false, incrementa sesion e interrumpe los hilos. Una tarea vieja ya encolada encuentra una sesión distinta y no actúa. Al reanudar se crean nuevos trabajadores usando los datos que permanecen en el modelo.

> “Los hilos son concurrentes, pero los cambios del modelo se atienden por turnos en el EDT. Los trabajadores despiertan cada 40 ms; eso no significa que produzcan cerebros cada 40 ms. La emisión depende del plazo aleatorio del motor.”

transient generadoresExternos: indica si la aplicación usa los trabajadores para revisar emisiones. No se serializa. El modo interno permite simulaciones y pruebas sin duplicar generadores cuando la interfaz está activa.

14. Pausa y guardado binario paso a paso

Relación con la rúbrica: Pausa completa, guardado por cuenta y recuperación pausada.

Pausar cambia tanto el modelo como los trabajadores

abrirPausa() pone pausada en true, detiene los hilos, deshabilita compras y abre el diálogo. Las operaciones del modelo también comprueban la pausa: no dependen solo del aspecto de los botones. Reanudar cambia la bandera, reinicia el instante de medición y crea trabajadores nuevos.

Fragmento real: GestorPartidas.java · líneas 25–35.

```
File destino = new File(carpeta, fecha + ".bin");
int sufijo = 1;
while (destino.exists()) destino = new File(carpeta, fecha + "-" + sufijo++ + ".bin");
File temporal = File.createTempFile("guardando-", ".tmp", carpeta);
try {
    try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(temporal))) {
        out.writeObject(new Estado(motor, fraccionSegundo));
    }
    if (!temporal.renameTo(destino)) throw new IOException("No se pudo completar el guardado.");
    return destino;
} finally { if (temporal.exists()) temporal.delete(); }
```

• **destino:** usa fecha/hora y extensión .bin. El while agrega un sufijo si ya existe otro con el mismo nombre.

• **temporal:** permite terminar la escritura antes de presentar el archivo como guardado completo.

• **ObjectOutputStream:** escribe un Estado que contiene motor y fraccionSegundo. Desde el motor se alcanzan la partida y sus entidades serializables.

• **try con recursos:** cierra el flujo incluso si ocurre una excepción. Tras escribir se renombra; finally limpia un temporal que haya quedado.

Qué se conserva y qué se reconstruye

Serializable permite guardar los campos de los objetos alcanzables: posiciones, salud, proyectiles, cerebros, podadoras, plazos internos y reloj. serialVersionUID identifica compatibilidad de serialización; Estado añade además una versión propia del formato.

cargar() usa ObjectInputStream y un filtro de clases y tamaño. Comprueba que haya un Estado válido, que el motor exista y la partida no esté terminada. vincularCuenta() exige el mismo nickname y la carga llama a pausar().

> “Se recuperan datos; no se recupera un Thread ejecutándose ni una ventana abierta. VentanaPartida se construye nuevamente con el motor recuperado y espera que el usuario pulse Reanudar.”

Prueba visible: guarda con un zombi o proyectil a mitad del tablero, vuelve al menú y carga. Compara tiempo, saldo y posiciones antes de continuar.

15. Estadísticas, archivo de texto y ranking

Relación con la rúbrica: Persistencia, porcentajes, promedio y tres Top 5.

Cuándo cambian los datos

Al finalizar, registrarVictoria(tiempo) o registrarDerrota() aumenta jugadas y el resultado correspondiente. Se agregan plantas destruidas, compras, gastos y podadoras de esa partida. Una victoria actualiza el mejor tiempo si es la primera o si fue más rápida.

Los resultados se guardan una sola vez mediante resultadoRegistrado. Guardar para continuar no se presenta como una victoria ni una derrota.

Cálculos

```
return (partidasGanadas * 100.0) / partidasJugadas;
return (double) plantasDestruidas / partidasJugadas;
```

Son las expresiones de getPorcentajeVictorias() y getPromedioPlantasDestruidas(); antes se comprueba si jugadas es cero. 100.0 y el cast double evitan la división entera que perdería decimales.

> “Con 3 victorias de 4 partidas, el porcentaje es 75%. Con 21 plantas destruidas en 4 partidas, el promedio es 5.25. Para rapidez se conserva el menor tiempo de una victoria, no el de una derrota.”

Cómo se persiste

escribirEstadisticas() escribe ocho líneas etiqueta: valor en estadisticas.txt. cargarEstadisticas() las lee en orden; leerValor() toma el texto después de los dos puntos y lo convierte a entero. El guardado comprobado escribe un temporal y reemplaza el destino.

Ordenamiento por inserción

Fragmento real: VentanaMenuPrincipal.java · líneas 77–81.

```
for (int i = 1; i < n; i++) {
    Cuenta c = ordenadas[i]; int j = i - 1;
    while (j >= 0 && comparar(c, ordenadas[j], tipo) < 0) { ordenadas[j + 1] = ordenadas[j]; j--;
        }
    ordenadas[j + 1] = c;
}
```

Se toma una cuenta desde la posición 1; mientras deba ir antes que la anterior, se desplaza esa anterior a la derecha. Después se coloca en el hueco. Por ejemplo, tiempos [40, 25, 35] quedan [25, 35, 40]. Se muestran como máximo cinco.

comparar() usa menor tiempo primero en rapidez, mayor cantidad primero en destrucción y victorias, y nickname para desempatar. El ranking de rapidez excluye cuentas sin victorias. El algoritmo opera sobre arreglos nativos.

16. Qué hacen las ventanas y el dibujo

Relación con la rúbrica: Swing, eventos, representación visual e interfaces.

Clase de Vista	Responsabilidad en el código
VentanaInicio	Presenta crear/seleccionar y abre la ventana correspondiente con el gestor.
VentanaCrearCuenta	Obtiene el texto, llama al gestor, muestra resultado y abre el menú con la cuenta creada.
VentanaSeleccionarCuenta	Llena el combo con las cuentas válidas; recupera la seleccionada y abre su menú.
VentanaMenuPrincipal	Navegación, lista de guardados, estadísticas y rankings.
VentanaNuevaPartida	JFileChooser, dificultad y creación de configuración, jardín, partida, motor y ventana.
VentanaPartida	Catálogo, etiquetas, acciones, reloj, pausa, guardado y resultado.
PanelJardinFondo	Transformación a píxeles, dibujo del tablero y entidades, selección de carril y recogida por clic.
HilosPartida	Ciclo de vida del reloj y trabajadores de generadores.
Recursos	Busca imágenes en rutas disponibles o dentro del JAR; las convierte y escala.
PanelFondo	Pinta un fondo escalado, lo centra y añade una capa oscura para legibilidad.

Cómo se convierte el clic en fila

```
int fila = (int) ((evento.getY() - area.y) / area.altoCasilla);
```

Se resta el borde superior del césped y se divide por la altura de una casilla. Con borde en 100, clic en 250 y casilla de 60 píxeles: (250 − 100) / 60 = 2.5; al convertir a int se obtiene la fila 2. Antes se comprueba que el clic esté dentro del césped.

Interfaz SeleccionCarril y representación

PanelJardinFondo informa la fila mediante alSeleccionarCarril(). La ventana registra una lambda que llama invocar(fila). El panel puede notificar sin conocer los detalles de compra.

repaint() solicita una actualización visual. Los métodos de dibujo leen posición y estado del modelo y seleccionan sprites, grietas, mensajes y barras. La imagen es una representación: cambiar un sprite no sustituye la lógica de salud o movimiento.

17. Consola y preguntas para defender el código

La consola no narra el combate

El juego es gráfico: saldo, tiempo, daño visual y resultado se muestran en Swing. La salida de NetBeans informa compilación, proceso Java y excepciones. BUILD SUCCESS o BUILD SUCCESSFUL corresponde al proceso de construcción o ejecución; no prueba por sí solo toda la funcionalidad.

GestorCuentas imprime algunos fallos de archivos. Los rechazos de compra o nivel se muestran mediante JOptionPane. Si se ejecutan las pruebas, aparecen sus mensajes OK; no se imprimen automáticamente al iniciar el juego.

Pregunta	Idea concreta para explicar
¿Qué significa this?	La referencia al objeto actual. this.cuenta = cuenta asigna el parámetro al atributo.
¿Qué significa super?	Invoca un constructor o una implementación de la clase padre.
¿Qué hace @Override?	Declara una redefinición de un método heredado; Java comprueba la relación.
¿Qué hacen return y continue?	return termina el método; continue pasa a la siguiente iteración del bucle. En la IA, return evita una segunda siembra.
¿Por qué null?	Representa ausencia de un objeto, como una casilla sin planta o una búsqueda sin cuenta.
¿Por qué instanceof?	Comprueba el tipo concreto antes de aplicar una regla especial o convertir la referencia.
¿Por qué final?	Impide reasignar esa variable. Si referencia un objeto, no implica que todo el objeto sea inmutable.
¿Por qué arreglos con contador?	La capacidad puede ser mayor que los elementos usados. El contador marca qué posiciones contienen datos activos.
¿Por qué serializar?	Para recuperar el grafo de datos y estados, no solo los valores que aparecen en las etiquetas.

Orden recomendado de demostración

Inicio → cuenta válida e inválida → selección → nivel JSON válido e inválido → compra y combate → recolección → pausa → guardar → recuperar → resultado → estadísticas. En cada paso muestra el método y explica entrada, condición y cambio de estado.

Documentación · 10 puntos

Muestra Manual_tecnico.pdf (UML, arquitectura, algoritmos, IA, hilos y archivos) y Manual_de_usuario.pdf (capturas del flujo). Esta guía es material de estudio para acompañar esos manuales. Practica siguiendo una acción completa entre clases, en lugar de memorizar únicamente los nombres de los métodos.

01. Proyecto2_PlantasVsZombies

Paquete: Principal · **Archivo:** principal/Proyecto2_PlantasVsZombies.java

Responsabilidad

Punto de entrada. Delega el arranque a Juego y utiliza la cola de eventos de Swing.

Atributos y relaciones

No conserva atributos ni reglas de combate. args es el arreglo de argumentos de la ejecución.

Un recorrido concreto

Java llama main; invokeLater recibe una lambda; el EDT construye Juego y ejecuta iniciar(). La ventana se abre por ese recorrido.

Pregunta para ensayar

¿Por qué main no tiene el combate? Porque delegar separa el arranque de las reglas y permite entender y probar cada responsabilidad.

Fragmento real de main() · inicio; los métodos largos se recortan aquí.

```
public static void main(String[] args) {

    javax.swing.SwingUtilities.invokeLater(() -> new Juego().iniciar());
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
main() Línea(s) 5	Programa en el EDT una lambda que construye Juego y llama iniciar(). No ejecuta aquí las reglas del juego.

02. Juego

Paquete: Principal · **Archivo:** principal/Juego.java

Responsabilidad

Coordina el inicio: prepara el gestor y abre la ventana inicial.

Atributos y relaciones

gestorCuentas referencia el servicio que carga y administra las cuentas.

Un recorrido concreto

new Juego() ejecuta su constructor y carga cuentas por medio de GestorCuentas. iniciar() crea VentanaInicio, la centra y la hace visible.

Pregunta para ensayar

¿Por qué se pasa el gestor a las ventanas? Para conservar el contexto de las cuentas mientras se navega.

Fragmento real de iniciar() · inicio; los métodos largos se recortan aquí.

```
public void iniciar() {

    VentanaInicio ventana =
            new VentanaInicio(gestorCuentas);

    ventana.setLocationRelativeTo(null);
    ventana.setVisible(true);
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Juego() Línea(s) 10	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
iniciar() Línea(s) 14	Crea VentanaInicio con el gestor, centra con setLocationRelativeTo(null) y muestra con setVisible(true).

03. RutasCuentas

Paquete: Archivos · **Archivo:** archivos/RutasCuentas.java

Responsabilidad

Traduce un nickname a una carpeta y recupera el nickname desde su nombre codificado.

Atributos y relaciones

No guarda estado de una cuenta; usa variables locales y métodos estáticos. El constructor es privado.

Un recorrido concreto

Ana se convierte en u-0041006e0061; ana en u-0061006e0061. Los nombres resultantes son distintos incluso en un disco que ignore mayúsculas.

Pregunta para ensayar

¿Por qué cuatro dígitos por carácter? Cada char de Java ocupa 16 bits y se representa con cuatro dígitos hexadecimales; los ceros conservan el ancho para decodificar.

Fragmento real de carpeta() · inicio; los métodos largos se recortan aquí.

```
    public static File carpeta(String nickname) {
        StringBuilder nombre = new StringBuilder("u-");
        for (int i = 0; i < nickname.length(); i++) {
            String hex = Integer.toHexString(nickname.charAt(i));
            for (int k = hex.length(); k < 4; k++) nombre.append('0');
            nombre.append(hex);
        }
        File codificada = new File("cuentas", nombre.toString());
        if (codificada.exists()) return codificada;
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
RutasCuentas() Línea(s) 7	Constructor privado: impide crear instancias de una utilidad que se utiliza mediante sus métodos estáticos.
carpeta() Línea(s) 8	Codifica el nickname en grupos hexadecimales de cuatro dígitos; prefiere esa carpeta y conserva compatibilidad con carpetas antiguas cuyo nombre coincida exactamente.
nickname() Línea(s) 23	Decodifica una carpeta con prefijo u-. Si no es codificada devuelve su nombre; un formato hexadecimal inválido produce cadena vacía.

04. GestorCuentas

Paquete: Archivos · **Archivo:** archivos/GestorCuentas.java

Responsabilidad

Valida, busca, crea y carga cuentas; lee y escribe sus estadísticas.

Atributos y relaciones

cuentas es un arreglo con capacidad inicial de 100. cantidadCuentas indica cuántas posiciones tienen cuentas; la capacidad puede crecer.

Un recorrido concreto

crearCuenta("Ana1") valida, busca duplicado, prepara archivos y agrega Cuenta al arreglo. Un duplicado devuelve false antes de crear otra cuenta.

Pregunta para ensayar

¿Qué diferencia hay entre capacidad y cantidad? El arreglo puede tener 100 posiciones y solo 3 cuentas; los recorridos útiles terminan en cantidadCuentas.

Fragmento real de crearCuenta() · inicio; los métodos largos se recortan aquí.

```
    public boolean crearCuenta(String nickname) {
        if (!nicknameValido(nickname) || buscarCuenta(nickname) != null) {
            return false;
        }

        ampliarSiNecesario();

        if (!crearArchivosCuenta(nickname)) {
            return false;
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
GestorCuentas() Línea(s) 17	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
nicknameValido() Línea(s) 23	Rechaza null o longitud fuera de 1–12; recorre caracteres con isLetterOrDigit y retorna false ante el primero inválido. No convierte a minúsculas.
buscarCuenta() Línea(s) 36	Recorre solo las cuentas utilizadas y compara nickname con equals; devuelve Cuenta o null.
crearCuenta() Línea(s) 45	Valida y evita duplicados, amplía el arreglo si hace falta, exige crear archivos y registra una nueva Cuenta; devuelve éxito o fallo.
crearArchivosCuenta() Línea(s) 61	Resuelve la ruta y crea partidas/ y estadisticas.txt. Un archivo nuevo se inicializa con ocho valores cero. Captura IOException y devuelve false ante fallo.
cargarCuentas() Línea(s) 81	Crea la raíz cuentas si falta; enumera carpetas, recupera nombres, evita duplicados y carga estadísticas antes de agregar cada Cuenta.
cargarEstadisticas() Línea(s) 106	Lee ocho líneas en orden mediante BufferedReader y construye Estadisticas. Si no existe el archivo o falla su lectura/conversión, retorna estadísticas en cero.

Método / ubicación	Explicación
leerValor() Línea(s) 132	Toma el texto posterior al último ':' y lo convierte con Integer.parseInt. Una línea ausente o sin separador devuelve cero; un número inválido lanza NumberFormatException.
guardarEstadisticas() Línea(s) 140	Envoltorio que llama al guardado comprobado y escribe un mensaje en System.err si no consigue guardar.
guardarEstadisticasComprobadas() Línea(s) 144	Escribe en un temporal dentro de la carpeta y usa Files.move con REPLACE_EXISTING. Devuelve booleano y limpia temporales en finally; no se solicita ATOMIC_MOVE.
ampliarSiNecesario() Línea(s) 158	Si cantidadCuentas alcanzó la capacidad, crea un arreglo del doble de tamaño y copia las referencias con System.arraycopy.
escribirEstadisticas() Línea(s) 164	Abre FileWriter y escribe ocho líneas etiqueta: valor; el try con recursos cierra el escritor.
getCuentas() Línea(s) 177	Consulta y devuelve cuentas. Si retorna un arreglo, es su referencia y no una copia defensiva; el llamador debe respetar el estado.
getCantidadCuentas() Línea(s) 178	Consulta y devuelve cantidadCuentas. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.

05. LectorNivel

Paquete: Archivos · **Archivo:** archivos/LectorNivel.java

Responsabilidad

Lee el archivo de nivel y lo convierte en datos validados mediante un analizador propio.

Atributos y relaciones

LectorNivel usa variables de lectura. La clase interna Analizador conserva texto y posicion, el cursor del análisis.

Un recorrido concreto

Al leer una fila de 4 plantas y otra de 3, leer() compara n con columnas y lanza un error. La ventana lo presenta como nivel inválido.

Pregunta para ensayar

¿Es un lector de cualquier JSON? No: implementa el esquema del proyecto, espera las claves en su orden y no admite escapes en cadenas. Debe explicarse ese alcance.

Fragmento real de tomar() · inicio; los métodos largos se recortan aquí.

```
private boolean tomar(char c) {
    espacios();
    if (posicion < texto.length() && texto.charAt(posicion) == c) { posicion++; return true; }
    return false;
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
leerNivel() Línea(s) 8	Lee UTF-8 por bloques, rechaza más de 4 000 000 caracteres y entrega todo a Analizador.leer(). El flujo se cierra automáticamente.
espacios() Línea(s) 22	Incrementa posicion mientras el carácter actual sea espacio en blanco; evita que saltos de línea y sangrías alteren el análisis.
tomar() Línea(s) 25	Salta espacios y consume un carácter solo si coincide; devuelve true o false. Sirve para separadores opcionales o repeticiones.
exigir() Línea(s) 30	Usa tomar; si no encuentra el símbolo requerido lanza un error con la posición actual.
error() Línea(s) 31	Construye IllegalArgumentException con mensaje y número de posición. El lanzamiento lo realiza quien la usa.
cadena() Línea(s) 34	Exige comillas, obtiene los caracteres hasta el cierre y devuelve substring. Rechaza escapes, controles y cadenas sin cerrar.
clave() Línea(s) 43	Lee una cadena, exige que sea el nombre esperado y luego exige ':'. Esto fija el orden de claves del formato.
entero() Línea(s) 47	Consume signo negativo opcional y dígitos; convierte con Integer.parseInt y transforma fallos de conversión en error de análisis.
valida() Línea(s) 79	Comprueba por igualdad si el identificador es VACIO o una de las siete plantas permitidas.
Analizador() Línea(s) 21	Constructor interno que conserva el texto; posicion comienza en cero, valor inicial de un int de instancia.

Método / ubicación	Explicación
leer() Línea(s) 54	Reconoce objeto, claves y matriz; valida números, identificadores y longitudes de filas; amplía arreglos temporales, exige cierre y devuelve ConfiguracionNivel de tamaño exacto.

06. GestorPartidas

Paquete: Archivos · **Archivo:** archivos/GestorPartidas.java

Responsabilidad

Guarda y recupera el modelo de una partida en binario y lista los archivos de una cuenta.

Atributos y relaciones

La clase interna Estado conserva version = 1, MotorCombate y fraccionSegundo. El gestor no conserva una ventana ni hilos.

Un recorrido concreto

Al guardar se exige pausa, se escribe un temporal con ObjectOutputStream y se renombra a fecha-hora.bin. Cargar valida el objeto y lo deja pausado.

Pregunta para ensayar

¿Por qué guardar el motor y no una captura? Una imagen no permite recuperar posiciones, salud, plazos de disparo, recursos ni estados del combate.

Fragmento real de guardar() · inicio; los métodos largos se recortan aquí.

```
    public File guardar(MotorCombate motor, double fraccionSegundo) throws IOException {
        Partida p = motor.getPartida();
        if (!p.estaPausada() || p.estaTerminada()) throw new IOException("Solo se guardan partidas
    pausadas y activas.");
        File carpeta = carpeta(p.getCuenta());
        if (!carpeta.isDirectory() && !carpeta.mkdirs()) throw new IOException("No se pudo crear la
    carpeta de partidas.");
        String fecha = new java.text.SimpleDateFormat("yyyy-MM-dd_HH-mm-ss-SSS").format(new
    java.util.Date());
        File destino = new File(carpeta, fecha + ".bin");
        int sufijo = 1;
        while (destino.exists()) destino = new File(carpeta, fecha + "-" + sufijo++ + ".bin");
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Estado() Línea(s) 13	Constructor de la clase interna serializable: agrupa el motor y la fracción de segundo. version identifica el formato del estado.
getMotor() Línea(s) 16	Consulta y devuelve motor. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getFraccionSegundo() Línea(s) 17	Consulta y devuelve fraccionSegundo. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
guardar() Línea(s) 19	Exige partida pausada y activa; resuelve carpeta por cuenta; crea nombre con fecha/hora, escribe Estado en temporal y lo renombra a .bin. Propaga IOException si no concluye.
cargar() Línea(s) 37	Configura filtro de objetos y límites; lee Estado, valida versión/motor/partida, exige la cuenta correspondiente y deja el modelo pausado. Convierte fallos de clase o cuenta en IOException.
listar() Línea(s) 60	Filtra archivos .bin de la cuenta y los ordena por inserción del nombre más reciente al más antiguo. Devuelve arreglo vacío si no puede listar.
carpeta() Línea(s) 71	Devuelve la subcarpeta partidas de la ruta específica de la Cuenta.

07. Cuenta

Paquete: Modelo · **Archivo:** modelo/Cuenta.java

Responsabilidad

Representa la identidad del jugador y su objeto de estadísticas.

Atributos y relaciones

nickname es String; estadisticas referencia Estadisticas. Implementa Serializable para formar parte del estado guardado.

Un recorrido concreto

El constructor con solo nickname crea estadísticas en cero; el constructor con Estadisticas permite reconstruir una cuenta desde su archivo.

Pregunta para ensayar

¿Cuenta maneja carpetas? No. Guarda datos; GestorCuentas y RutasCuentas resuelven la persistencia.

Fragmento real de Cuenta() · inicio; los métodos largos se recortan aquí.

```
public Cuenta(String nickname) {
    this.nickname = nickname;
    this.estadisticas = new Estadisticas();
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Cuenta() Línea(s) 9, 14 Sobrecargas o callbacks agrupados	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super. Hay 2 sobrecargas, para distintas formas de inicialización.
getNickname() Línea(s) 19	Consulta y devuelve nickname. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getEstadisticas() Línea(s) 23	Consulta y devuelve estadisticas. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.

08. Estadisticas

Paquete: Modelo · **Archivo:** modelo/Estadisticas.java

Responsabilidad

Conserva resultados acumulados y calcula porcentaje de victorias y promedio de plantas.

Atributos y relaciones

Ocho enteros: jugadas, ganadas, perdidas, plantasDestruidas, zombisInvocados, cerebrosGastados, mejorTiempoVictoria y podadorasActivadas.

Un recorrido concreto

Si una cuenta gana por primera vez en 40 s, se guarda 40. Ganar luego en 25 lo reemplaza por 25; ganar en 35 mantiene 25.

Pregunta para ensayar

¿Por qué comprobar jugadas antes de dividir? Evita división por cero. El 100.0 o el cast double conserva resultados decimales.

Fragmento real de registrarVictoria() · inicio; los métodos largos se recortan aquí.

```
public void registrarVictoria(int tiempo) {
    partidasJugadas++;
    partidasGanadas++;
    if (partidasGanadas == 1 || tiempo < mejorTiempoVictoria) {
        mejorTiempoVictoria = tiempo;
    }
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Estadisticas() Línea(s) 15, 19 Sobrecargas o callbacks agrupados	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super. Hay 2 sobrecargas, para distintas formas de inicialización.
registrarVictoria() Línea(s) 32	Incrementa jugadas y ganadas. Conserva el tiempo si es la primera victoria o si mejora el menor tiempo anterior; una victoria de cero segundos se conserva correctamente.
registrarDerrota() Línea(s) 40	Incrementa jugadas y perdidas sin modificar el mejor tiempo de victoria.
sumarPlantasDestruidas() Línea(s) 45	Acumula la cantidad de plantas de una partida finalizada.
sumarZombisInvocados() Línea(s) 46	Acumula las compras del jugador, no las entidades generadas gratis.
sumarCerebrosGastados() Línea(s) 47	Acumula el precio realmente pagado, incluidos descuentos.
sumarPodadorasActivadas() Línea(s) 48	Acumula las podadoras consumidas en las partidas terminadas.
getPorcentajeVictorias() Línea(s) 50	Retorna cero si no hay partidas; en otro caso ganadas × 100.0 / jugadas para mantener decimales.

Método / ubicación	Explicación
getPromedioPlantasDestruidas () Línea(s) 55	Evita dividir entre cero y usa conversión a double para devolver plantas / partidas con decimales.
getPartidasJugadas() Línea(s) 60	Consulta y devuelve partidasJugadas. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getPartidasGanadas() Línea(s) 61	Consulta y devuelve partidasGanadas. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getPartidasPerdidas() Línea(s) 62	Consulta y devuelve partidasPerdidas. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getPlantasDestruidas() Línea(s) 63	Consulta y devuelve plantasDestruidas. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getZombisInvocados() Línea(s) 64	Consulta y devuelve zombisInvocados. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getCerebrosGastados() Línea(s) 65	Consulta y devuelve cerebrosGastados. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getMejorTiempoVictoria() Línea(s) 66	Consulta y devuelve mejorTiempoVictoria. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getPodadorasActivadas() Línea(s) 67	Consulta y devuelve podadorasActivadas. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.

09. ConfiguracionNivel

Paquete: Modelo · **Archivo:** modelo/ConfiguracionNivel.java

Responsabilidad

Transporta los datos iniciales validados entre la lectura del archivo y la construcción del juego.

Atributos y relaciones

duracionSegundos y cerebrosIniciales son enteros; jardin es String[][]. Las dimensiones se consultan sobre esa matriz.

Un recorrido concreto

Un archivo de 180 s y matriz 5 × 9 produce una configuración; Jardin transforma los identificadores en entidades, y Partida copia el saldo y duración.

Pregunta para ensayar

¿Es el estado actual de la partida? No: representa la configuración inicial. El saldo y reloj que cambian se conservan en Partida.

Fragmento real de ConfiguracionNivel() · inicio; los métodos largos se recortan aquí.

```
public ConfiguracionNivel(
        int duracionSegundos,
        int cerebrosIniciales,
        String[][] jardin) {

    this.duracionSegundos = duracionSegundos;
    this.cerebrosIniciales = cerebrosIniciales;
    this.jardin = jardin;
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
ConfiguracionNivel() Línea(s) 10	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getDuracionSegundos() Línea(s) 20	Consulta y devuelve duracionSegundos. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getCerebrosIniciales() Línea(s) 24	Consulta y devuelve cerebrosIniciales. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getJardin() Línea(s) 28	Consulta y devuelve jardin. Si retorna un arreglo, es su referencia y no una copia defensiva; el llamador debe respetar el estado.
getFilas() Línea(s) 32	Consulta y devuelve jardin.length. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getColumnas() Línea(s) 36	Consulta y devuelve jardin[0].length. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.

10. Casilla

Paquete: Modelo · **Archivo:** modelo/Casilla.java

Responsabilidad

Representa una ubicación del jardín con cero o una planta.

Atributos y relaciones

fila y columna ubican la celda; planta es una referencia que puede ser null.

Un recorrido concreto

Una casilla vacía tiene planta == null. colocarPlanta(p) guarda la referencia; quitarPlanta() vuelve a null sin borrar la casilla.

Pregunta para ensayar

¿Se borra la fila al destruir una planta? No. Se retira la planta de esa celda y las dimensiones permanecen iguales.

Fragmento real de Casilla() · inicio; los métodos largos se recortan aquí.

```
public Casilla(int fila, int columna) {

    this.fila = fila;
    this.columna = columna;
    this.planta = null;
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Casilla() Línea(s) 12	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getFila() Línea(s) 19	Consulta y devuelve fila. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getColumna() Línea(s) 23	Consulta y devuelve columna. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getPlanta() Línea(s) 27	Consulta y devuelve planta. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
estaVacia() Línea(s) 31	Devuelve si planta == null. La ausencia de planta no implica ausencia de Casilla.
colocarPlanta() Línea(s) 35	Asigna la referencia recibida; la validación de siembra se hace en Jardin.sembrar().
quitarPlanta() Línea(s) 39	Asigna null para dejar la casilla libre.

11. Jardin

Paquete: Modelo · **Archivo:** modelo/Jardin.java

Responsabilidad

Construye la matriz de casillas y crea plantas según los identificadores; permite nuevas siembras válidas.

Atributos y relaciones

Casilla[][] casillas; enteros filas y columnas. Las dimensiones vienen del arreglo validado, no de una constante fija.

Un recorrido concreto

Los for recorren cada fila y columna, crean Casilla y llaman crearPlanta(). VACIO devuelve null; NUEZ construye Nuez en esa coordenada.

Pregunta para ensayar

¿Por qué dos for? Uno recorre filas y el otro columnas. En una matriz de F por C se visitan F × C posiciones.

Fragmento real de crearJardin() · inicio; los métodos largos se recortan aquí.

```
    private void crearJardin(String[][] configuracion) {

        for (int fila = 0; fila < filas; fila++) {

            for (int columna = 0; columna < columnas; columna++) {

                casillas[fila][columna] =
                        new Casilla(fila, columna);

// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Jardin() Línea(s) 19	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
crearJardin() Línea(s) 29	Dos for crean cada Casilla y su planta según el String[][]; solo colocan planta si crearPlanta no devolvió null.
crearPlanta() Línea(s) 54	Relaciona identificadores con constructores concretos; VACIO devuelve null. La validación del conjunto permitido se realiza previamente en LectorNivel.
getCasilla() Línea(s) 94	Consulta y devuelve casillas[fila][columna]. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getCasillas() Línea(s) 98	Consulta y devuelve casillas. Si retorna un arreglo, es su referencia y no una copia defensiva; el llamador debe respetar el estado.
getFilas() Línea(s) 102	Consulta y devuelve filas. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getColumnas() Línea(s) 106	Consulta y devuelve columnas. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
sembrar() Línea(s) 110	Valida límites y casilla vacía, construye la planta y la coloca. Devuelve false si el tipo no produce planta o la ubicación es inválida.

12. Partida

Paquete: Modelo · **Archivo:** modelo/Partida.java

Responsabilidad

Conserva el estado general y protege operaciones como comprar, pausar y consumir podadoras.

Atributos y relaciones

Cuenta, Jardin y dificultad; duración y reloj; saldo y gastos; arreglo de zombis y contador; pausa/final; podadoras disponibles por fila y número de activaciones; fracción de reloj.

Un recorrido concreto

Con 150 cerebros en periodo normal, comprar Caracono deja 50, suma 100 al gasto y una compra. Un segundo Caracono no se agrega si no alcanza el saldo.

Pregunta para ensayar

¿Por qué se comprueba pausa también en el modelo? Deshabilitar botones no basta: el estado debe impedir cambios aunque otra llamada intente comprar o recoger recursos.

Fragmento real de invocarZombi() · inicio; los métodos largos se recortan aquí.

```
    public boolean invocarZombi(Zombi zombi) {
        if (zombi == null || pausada || terminada
                || zombi.getCosto() <= 0
                || zombi.getFila() < 0
                || zombi.getFila() >= jardin.getFilas()
                || costoActual(zombi.getCosto()) > cerebros) {
            return false;
        }

// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Partida() Línea(s) 30	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getCuenta() Línea(s) 54	Consulta y devuelve cuenta. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getJardin() Línea(s) 58	Consulta y devuelve jardin. Si retorna un arreglo, es su referencia y no una copia defensiva; el llamador debe respetar el estado.
getDificultad() Línea(s) 62	Consulta y devuelve dificultad. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getCerebros() Línea(s) 66	Consulta y devuelve cerebros. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getTiempoRestante() Línea(s) 70	Consulta y devuelve tiempoRestante. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getCantidadZombis() Línea(s) 74	Consulta y devuelve cantidadZombis. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getZombi() Línea(s) 78	Consulta y devuelve zombis[indice]. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.

Método / ubicación	Explicación
estaPausada() Línea(s) 85	Consulta la bandera que impide avanzar e interactuar durante la pausa.
estaTerminada() Línea(s) 89	Consulta si el modelo ya no debe continuar el juego.
pausar() Línea(s) 93	Activa pausada solo si la partida no terminó; detener trabajadores lo realiza la ventana.
reanudar() Línea(s) 99	Desactiva pausada solo si no terminó; la ventana coordina el reinicio de trabajadores.
terminar() Línea(s) 105	Activa terminada para impedir nuevas operaciones de juego.
tienePodadora() Línea(s) 110	Comprueba fila válida y disponibilidad del booleano de ese carril.
activarPodadora() Línea(s) 116	Rechaza pausa, final o ausencia; cambia disponibilidad a false y suma una activación.
getPodadorasActivadas() Línea(s) 123	Consulta y devuelve podadorasActivadas. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
descontarUnSegundo() Línea(s) 127	Reduce el reloj únicamente si la partida está activa y el tiempo es positivo.
agregarCerebros() Línea(s) 133	Suma solo cantidades positivas durante una partida activa.
invocarZombi() Línea(s) 143	Valida referencia, estado, costo positivo, fila y fondos; agrega el zombi, descuenta costoActual y suma gastos y compras.
agregarZombiGenerado() Línea(s) 161	Valida estado y fila pero agrega sin cobrar ni incrementar las compras: se usa para bailarines y Zombidito.
guardarEnArreglo() Línea(s) 171	Duplica capacidad si se llenó, copia referencias y escribe al final utilizado; incrementa cantidadZombis.
vincularCuenta() Línea(s) 183	Exige mismo nickname que la cuenta del guardado y sustituye la referencia por la cuenta actual.
setFraccionReloj() Línea(s) 188	Acepta únicamente valores entre cero incluido y uno excluido para calcular el bloque sin perder fracciones.
getDuracionTotal() Línea(s) 191	Consulta y devuelve duracionTotal. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getZombisComprados() Línea(s) 192	Consulta y devuelve zombisComprados. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getCerebrosGastados() Línea(s) 193	Consulta y devuelve cerebrosGastados. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getBloque() Línea(s) 194	Calcula el tiempo transcurrido, lo transforma a siete periodos y limita el resultado a 7.
costoActual() Línea(s) 199	Cobra un cuarto en bloque 7, mitad en bloque 4 y base en el resto. La división entera con suma previa redondea hacia arriba.

13. Proyectil

Paquete: Modelo · **Archivo:** modelo/Proyectil.java

Responsabilidad

Representa un guisante en vuelo con dirección, daño y efecto.

Atributos y relaciones

fila; posición x; direccion −1 o +1; danio; hielo; velocidad fija de 3 columnas por segundo.

Un recorrido concreto

Con x = 2, dirección −1 y dt = 0.1, avanzar() resta 0.3 y deja x = 1.7. MotorCombate decide si chocó o salió del tablero.

Pregunta para ensayar

¿Por qué usar columnas y no píxeles? El movimiento lógico no cambia cuando el panel se redimensiona; la vista convierte a píxeles.

Fragmento real de avanzar() · inicio; los métodos largos se recortan aquí.

```
public void avanzar(double segundos) { x += direccion * velocidad * segundos; }
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Proyectil() Línea(s) 17	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getFila() Línea(s) 26	Consulta y devuelve fila. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getX() Línea(s) 27	Consulta y devuelve x. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getDireccion() Línea(s) 28	Consulta y devuelve direccion. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getDanio() Línea(s) 29	Consulta y devuelve danio. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
esHielo() Línea(s) 30	Consulta si el impacto debe renovar la ralentización además de causar daño.
avanzar() Línea(s) 31	Actualiza x sumando direccion × velocidad × segundos. El motor resuelve colisión y salida del jardín.

14. Cerebro

Paquete: Modelo · **Archivo:** modelo/Cerebro.java

Responsabilidad

Representa un recurso visible pendiente de recoger, no el saldo numérico del jugador.

Atributos y relaciones

x/y son coordenadas lógicas; vence es el instante de caducidad del reloj del motor; generador identifica el origen. −1 indica una planta destruida.

Un recorrido concreto

Creado cuando ahora vale 12 con vence = 17: permanece hasta el segundo 17 de juego. Recogerlo antes entrega 25; después no entrega recursos.

Pregunta para ensayar

¿Por qué almacenar el origen? Cuando desaparece un icono de un generador se puede programar su siguiente espera. Los de plantas no reprogramen generadores.

Fragmento real de Cerebro() · inicio; los métodos largos se recortan aquí.

```
public Cerebro(double x, double y, double vence, int generador) {
    this.x = x; this.y = y; this.vence = vence; this.generador = generador;
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Cerebro() Línea(s) 8	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getX() Línea(s) 11	Consulta y devuelve x. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getY() Línea(s) 12	Consulta y devuelve y. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getVence() Línea(s) 13	Consulta y devuelve vence. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getGenerador() Línea(s) 14	Consulta y devuelve generador. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.

15. MotorCombate

Paquete: Modelo · **Archivo:** modelo/MotorCombate.java

Responsabilidad

Coordina la evolución del combate, economía, defensa y efectos de entidades.

Atributos y relaciones

Referencias a Partida/Jardin; arreglos de proyectiles y cerebros con contadores; matrices de plazos por planta; arreglos de efectos por zombi; reloj ahora, Random, defensa siguiente, victoria y animaciones. generadoresExternos es transient.

Un recorrido concreto

actualizar(0.04) incrementa ahora y coordina economía, IA, preparación, disparos, colisiones, movimiento y reglas por casilla. Una llamada durante pausa retorna sin avanzar.

Pregunta para ensayar

¿Por qué hay arreglos paralelos por zombi? El índice del zombi permite consultar sus plazos de ataque, hielo, baile y vuelo sin usar colecciones. Deben ampliarse juntos.

Fragmento real de actualizar() · inicio; los métodos largos se recortan aquí.

```
    public void actualizar(double segundos) {
        if (segundos <= 0 || partida.estaPausada() || partida.estaTerminada()) return;
        segundos = Math.min(segundos, 0.15);
        ahora += segundos;
        actualizarEconomia();
        actualizarDefensa();
        for (int f = 0; f < animacionPodadoras.length; f++) {
            if (animacionPodadoras[f] > 0) {
                animacionPodadoras[f] = Math.max(0, animacionPodadoras[f] - segundos);
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
MotorCombate() Línea(s) 45	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getCantidadProyectiles() Línea(s) 69	Consulta y devuelve cantidadProyectiles. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getProyectil() Línea(s) 70	Consulta y devuelve proyectiles[i]. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getPlantasDestruidas() Línea(s) 71	Consulta y devuelve plantasDestruidas. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getTiempoTranscurrido() Línea(s) 72	Consulta y devuelve ahora. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
hayVictoria() Línea(s) 73	Consulta la bandera de invasión; VentanaPartida la usa para registrar el resultado.
tienePodadora() Línea(s) 74	Delega la consulta de disponibilidad al estado de Partida.
getPodadorasActivadas() Línea(s) 75	Consulta y devuelve partida.getPodadorasActivadas(). Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.

Método / ubicación	Explicación
progresoPodadora() Línea(s) 78	Devuelve −1 si no hay barrido activo o el progreso normalizado entre 0 y 1 durante la animación de 0.9 s.
estaCongelado() Línea(s) 83	Encuentra el índice del objeto y compara ahora con congeladoHasta, comprobando antes los límites del arreglo.
indiceZombi() Línea(s) 88	Busca la misma referencia con == en el arreglo de la partida; devuelve −1 si no la encuentra. Es identidad del objeto, no igualdad de nickname.
actualizar() Línea(s) 96	Rechaza paso no positivo, pausa y final; limita dt, incrementa ahora y coordina economía, defensa, plazos, disparos, proyectiles, zombis y efectos de casilla.
asegurarEspacioZombis() Línea(s) 115	Amplía juntos los arreglos paralelos de mordisco, hielo, baile, salto, lanzamiento y vuelo. Inicializa arreglos de vuelo ausentes en guardados previos.
activarPatatapumsYDigestion() Línea(s) 144	Recorre plantas: activa Patatapum al cumplir preparadaEn y termina digestión cuando vence digestionHasta.
dispararPlantas() Línea(s) 164	Recorre plantas vivas listas; usa instanceof para disparadores, busca blancos por dirección, crea proyectiles y programa proximoDisparo. Bipetidora consulta dos direcciones.
hayZombiEnDireccion() Línea(s) 203	Busca un zombi vivo y no en vuelo en el mismo carril; compara su centro con la planta y la dirección pedida.
agregarProyectil() Línea(s) 214	Amplía el arreglo si hace falta y agrega el proyectil al final utilizado.
quitarProyectil() Línea(s) 223	Desplaza elementos posteriores una posición a la izquierda, reduce el contador y limpia la última referencia usada.
moverProyectiles() Línea(s) 228	Avanza cada proyectil y revisa todo el tramo recorrido con tolerancia; elige el primer blanco del carril, aplica daño, renueva hielo a ahora + 4 y retira proyectil. También retira los que salen.
moverYAtacarZombis() Línea(s) 260	Recorre vivos, resuelve lanzamiento y baile, busca el obstáculo frontal, calcula avance con velocidad/hielo, salta o ataca/aplasta y comprueba llegada a la casa. Omite entidades en vuelo.
llegadaALaCasa() Línea(s) 354	Si hay podadora, la consume y elimina todos los zombis vivos de la fila respetando recibirDanio polimórfico; inicia barrido visual. Si ya se usó, marca victoria y termina.
invocarBailarines() Línea(s) 374	Revisa arriba, abajo, izquierda y derecha de la casilla del Bailón. Exige límites y ausencia de planta viva o zombi vivo en la celda; agrega ZombiBailarin sin cobrar.
atacarConMordisco() Línea(s) 398	Exige planta viva y plazo de ataque cumplido; aplica daño y programa 1 s o 0.5 s para Deportista. Si la planta muere la retira y detiene el ataque.
destruirPlanta() Línea(s) 410	Retira la planta de la casilla, incrementa destrucciones y crea 3–6 cerebros con vigencia de 5 s y origen −1.
pinchohierbaYPatatapum() Línea(s) 421	Resuelve daño local de Pinchohierba, explosión de Patatapum y engullir/daño y digestión de Carroñívora. Usa casillaActual y excluye zombis en vuelo.
casillaActual() Línea(s) 473	Calcula floor(posicionX + 0.35) y limita al intervalo válido de columnas; fija una casilla lógica única para ataques de área.
getPartida() Línea(s) 478	Consulta y devuelve partida. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.

Método / ubicación	Explicación
getCantidadCerebros() Línea(s) 479	Consulta y devuelve cantidadCerebros. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getCerebro() Línea(s) 480	Consulta y devuelve cerebros[i]. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getCantidadGeneradores() Línea(s) 481	Consulta y devuelve proximoCerebro.length. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
estaSaltando() Línea(s) 482	Consulta el plazo visual saltoHasta del zombi, sin convertir el movimiento completo en un nuevo hilo.
estaLanzando() Línea(s) 483	Consulta si sigue vigente el plazo de la animación de lanzamiento del gigante.
recienAplastada() Línea(s) 484	Consulta la matriz aplastadaHasta para dibujar el efecto de una planta aplastada.
estaEnVuelo() Línea(s) 485	Comprueba índice, existencia del arreglo y plazo vueloHasta; evita ataques y colisiones durante el vuelo de Zombidito.
posicionVisual() Línea(s) 489	Interpola del origen al destino de Zombidito durante 0.8 s; fuera del vuelo devuelve la posición lógica normal.
alturaVuelo() Línea(s) 494	Devuelve sin(pi × progreso) × 0.9 durante el vuelo; produce un arco que empieza y termina a altura cero.
esperaCerebro() Línea(s) 499	Devuelve 5 + nextInt(11): un entero entre 5 y 15 segundos incluidos.
agregarCerebro() Línea(s) 500	Amplía el arreglo de iconos si hace falta y agrega el nuevo objeto al tramo utilizado.
quitarCerebro() Línea(s) 507	Si provino de un generador programa su siguiente espera; desplaza iconos, baja el contador y limpia la referencia final.
recogerCerebro() Línea(s) 513	Rechaza pausa, final e índice inválido; retira un vencido sin premio, o retira uno vigente y agrega 25 al saldo.
actualizarEconomia() Línea(s) 518	Retira iconos vencidos. Si no hay trabajadores externos activos, evalúa generadores internamente para simulaciones y pruebas.
usarGeneradoresExternos() Línea(s) 526	Cambia una bandera transient que evita evaluar emisiones por dos vías cuando la interfaz tiene trabajadores.
actualizarGenerador() Línea(s) 527	Si la partida y el índice son válidos y vence el plazo, crea un icono en posición aleatoria y fija el siguiente plazo en infinito hasta que desaparezca.
sembrar() Línea(s) 536	Solicita Jardin.sembrar; si tiene éxito reinicia plazos de preparación, disparo, pinchos y digestión de esa celda.
sembrarEnCarril() Línea(s) 544	Busca espacios desde la casilla posterior al zombi hasta el extremo de la casa; retorna al conseguir una siembra.
actualizarDefensa() Línea(s) 550	Controla intervalo y dificultad. Medio elige tipo aleatorio y busca desde casa. Difícil revisa prioridades globales y retorna después de una acción exitosa.

16. Planta

Paquete: Plantas · **Archivo:** plantas/Planta.java

Responsabilidad

Clase abstracta de los elementos defensores; reúne salud y posición.

Atributos y relaciones

salud es mutable y privada; fila y columna son final. serialVersionUID identifica la serialización.

Un recorrido concreto

Con 100 de salud, recibirDanio(20) deja 80. Si recibe luego 100, se limita a cero y estaViva() devuelve false.

Pregunta para ensayar

¿Puede crearse new Planta(...)? No, es abstracta. Sus constructores se ejecutan como parte de la construcción de las hijas mediante super.

Fragmento real de recibirDanio() · inicio; los métodos largos se recortan aquí.

```
public void recibirDanio(int danio) {

    salud = salud - danio;

    if (salud < 0) {
        salud = 0;
    }
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Planta() Línea(s) 10	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getSalud() Línea(s) 16	Consulta y devuelve salud. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getFila() Línea(s) 20	Consulta y devuelve fila. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getColumna() Línea(s) 24	Consulta y devuelve columna. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
recibirDanio() Línea(s) 28	Resta la cantidad a salud y limita a cero para evitar salud negativa.
estaViva() Línea(s) 37	Devuelve salud > 0; el motor y el pintado omiten una planta sin vida.

17. Lanzaguisantes

Paquete: Plantas · **Archivo:** plantas/Lanzaguisantes.java

Responsabilidad

Define una planta de disparo frontal; el motor crea sus proyectiles.

Atributos y relaciones

Hereda 100 HP y posición. danio = 20 y cadenciaDisparo = 1000 ms son final.

Un recorrido concreto

MotorCombate comprueba blanco frontal y programa disparos. No hay un Thread en esta clase: los tiempos se comparan en el ciclo del motor.

Pregunta para ensayar

¿El getter dispara? No; retorna el dato. dispararPlantas() del motor consulta esos datos y crea Proyectil.

Fragmento real de Lanzaguisantes() · inicio; los métodos largos se recortan aquí.

```
public Lanzaguisantes(int fila, int columna) {

    super(100, fila, columna);

    danio = 20;
    cadenciaDisparo = 1000;
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Lanzaguisantes() Línea(s) 8	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getDanio() Línea(s) 16	Consulta y devuelve danio. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getCadenciaDisparo() Línea(s) 20	Consulta y devuelve cadenciaDisparo. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.

18. Patatapum

Paquete: Plantas · **Archivo:** plantas/Patatapum.java

Responsabilidad

Define la mina y conserva si ya terminó su preparación.

Atributos y relaciones

50 HP heredados, danio = 100, tiempoPreparacion = 10000 ms y activa inicialmente false.

Un recorrido concreto

Con preparadaEn = 10, al llegar ahora a 10 el motor llama activar(). Cuando coincide al menos un zombi en su casilla aplica daño a todos los afectados y la retira.

Pregunta para ensayar

¿Una mina sembrada al segundo 30 nace lista? No: el motor asigna preparadaEn = ahora + 10 para esa celda.

Fragmento real de Patatapum() · inicio; los métodos largos se recortan aquí.

```
public Patatapum(int fila, int columna) {

    super(50, fila, columna);

    danio = 100;
    tiempoPreparacion = 10000;
    activa = false;
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Patatapum() Línea(s) 9	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getDanio() Línea(s) 18	Consulta y devuelve danio. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getTiempoPreparacion() Línea(s) 22	Consulta y devuelve tiempoPreparacion. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
estaActiva() Línea(s) 26	Consulta la bandera de preparación; la explosión se decide en el motor.
activar() Línea(s) 30	Pone activa = true cuando el motor determina que venció el plazo de preparación.

19. Hielaguisantes

Paquete: Plantas · **Archivo:** plantas/Hielaguisantes.java

Responsabilidad

Define la planta que produce proyectiles con efecto de ralentización.

Atributos y relaciones

100 HP; danio = 15; cadencia = 1500 ms; duracionCongelamiento = 4000 ms y reduccionVelocidad = 0.40.

Un recorrido concreto

Un impacto a ahora = 12 fija congeladoHasta = 16; otro a 14 lo cambia a 18. La velocidad sigue al 60%, no al 36%.

Pregunta para ensayar

¿Todo getter de esta clase controla directamente el motor? No. El motor consulta daño y cadencia, pero la duración de 4 s y factor 0.60 están expresados también como constantes en su lógica.

Fragmento real de Hielaguisantes() · inicio; los métodos largos se recortan aquí.

```
public Hielaguisantes(int fila, int columna) {

    super(100, fila, columna);

    danio = 15;
    cadenciaDisparo = 1500;
    duracionCongelamiento = 4000;
    reduccionVelocidad = 0.40;
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Hielaguisantes() Línea(s) 10	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getDanio() Línea(s) 20	Consulta y devuelve danio. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getCadenciaDisparo() Línea(s) 24	Consulta y devuelve cadenciaDisparo. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getDuracionCongelamiento() Línea(s) 28	Consulta y devuelve duracionCongelamiento. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getReduccionVelocidad() Línea(s) 32	Consulta y devuelve reduccionVelocidad. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.

20. Nuez

Paquete: Plantas · **Archivo:** plantas/Nuez.java

Responsabilidad

Representa una planta resistente que bloquea y cambia de estado visual.

Atributos y relaciones

400 HP heredados; saludMaxima = 400 se usa para calcular el porcentaje.

Un recorrido concreto

Con 200 de salud: 200 × 100 / 400 = 50%, por lo que devuelve estado 2. Con 100: 25%, estado 3. El panel dibuja las grietas.

Pregunta para ensayar

¿obtenerEstadoVisual() cambia la imagen? Devuelve un código; PanelJardinFondo interpreta ese valor y realiza el dibujo.

Fragmento real de obtenerEstadoVisual() · inicio; los métodos largos se recortan aquí.

```
    public int obtenerEstadoVisual() {

        int porcentaje = obtenerPorcentajeSalud();

        if (porcentaje >= 66) {
            return 1;
        }

        if (porcentaje >= 35) {
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Nuez() Línea(s) 7	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getSaludMaxima() Línea(s) 14	Consulta y devuelve saludMaxima. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
obtenerPorcentajeSalud() Línea(s) 18	Calcula salud × 100 / saludMaxima con enteros: devuelve un porcentaje entero.
obtenerEstadoVisual() Línea(s) 23	Devuelve 1 desde 66%, 2 desde 35% y 3 por debajo de 35%; no pinta directamente.

21. Carronivora

Paquete: Plantas · **Archivo:** plantas/Carronivora.java

Responsabilidad

Conserva la digestión y los parámetros para engullir o atacar al gigante.

Atributos y relaciones

100 HP; danioContraZombistein = 200; tiempoDigestion = 8000 ms; digiriendo inicia en false.

Un recorrido concreto

Al comer, iniciarDigestion() pone true; el motor guarda ahora + 8. Hasta vencer ese plazo no vuelve a engullir; terminarDigestion() pone false.

Pregunta para ensayar

¿Qué ocurre con un escudo al engullir? El motor suma salud y escudo para aplicar daño suficiente. A Zombistein le aplica 200, sin engullirlo.

Fragmento real de Carronivora() · inicio; los métodos largos se recortan aquí.

```
public Carronivora(int fila, int columna) {

    super(100, fila, columna);

    danioContraZombistein = 200;
    tiempoDigestion = 8000;
    digiriendo = false;
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Carronivora() Línea(s) 9	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getDanioContraZombistein() Línea(s) 18	Consulta y devuelve danioContraZombistein. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getTiempoDigestion() Línea(s) 22	Consulta y devuelve tiempoDigestion. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
estaDigiriendo() Línea(s) 26	Consulta si está bloqueada temporalmente para engullir de nuevo.
iniciarDigestion() Línea(s) 30	Activa digiriendo; el motor conserva el plazo de ocho segundos.
terminarDigestion() Línea(s) 34	Desactiva digiriendo cuando el motor comprueba que venció el plazo.

22. Pinchohierba

Paquete: Plantas · **Archivo:** plantas/Pinchohierba.java

Responsabilidad

Define daño repetido por casilla y evita mordiscos normales.

Atributos y relaciones

Salud heredada 1 como representación interna; danio = 10 e intervaloDanio = 500 ms.

Un recorrido concreto

recibirDanio() está sobrescrito y no resta salud. El motor omite esta planta como obstáculo y Zombistein puede retirarla directamente de la casilla.

Pregunta para ensayar

¿Por qué el método recibirDanio está vacío? Es una redefinición intencional para impedir daño normal; la destrucción por aplastamiento se realiza por otra operación.

Fragmento real de recibirDanio() · inicio; los métodos largos se recortan aquí.

```
public void recibirDanio(int danio) {
    // Los zombis normales no pueden dañarla.
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Pinchohierba() Línea(s) 8	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getDanio() Línea(s) 16	Consulta y devuelve danio. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getIntervaloDanio() Línea(s) 20	Consulta y devuelve intervaloDanio. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
recibirDanio() Línea(s) 25	Redefine el método heredado sin restar salud para ignorar ataques normales; el motor resuelve el aplastamiento retirándola de la celda.

23. Bipetidora

Paquete: Plantas · **Archivo:** plantas/Bipetidora.java

Responsabilidad

Define la planta con disparo frontal simple y trasero doble.

Atributos y relaciones

100 HP; danio = 15; ambas cadencias = 1000 ms y disparosTraseros = 2.

Un recorrido concreto

Si hay un zombi a cada lado, el motor crea tres proyectiles: uno con dirección −1 y dos con +1. Sin blanco de un lado no crea los de ese lado.

Pregunta para ensayar

¿Usa dos relojes independientes? En este código el motor utiliza proximoDisparo y la cadencia frontal para ambas direcciones, cuyos parámetros valen lo mismo.

Fragmento real de Bipetidora() · inicio; los métodos largos se recortan aquí.

```
public Bipetidora(int fila, int columna) {

    super(100, fila, columna);

    danio = 15;
    cadenciaFrontal = 1000;
    cadenciaTrasera = 1000;
    disparosTraseros = 2;
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Bipetidora() Línea(s) 10	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getDanio() Línea(s) 20	Consulta y devuelve danio. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getCadenciaFrontal() Línea(s) 24	Consulta y devuelve cadenciaFrontal. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getCadenciaTrasera() Línea(s) 28	Consulta y devuelve cadenciaTrasera. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getDisparosTraseros() Línea(s) 32	Consulta y devuelve disparosTraseros. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.

24. Zombi

Paquete: Zombis · **Archivo:** zombis/Zombi.java

Responsabilidad

Clase abstracta de los atacantes; conserva estado común y movimiento básico.

Atributos y relaciones

Salud actual/máxima, daño, costo, modificadorVelocidad, fila, posicionX y atacando. Los setters protected permiten ajustes desde hijas.

Un recorrido concreto

mover(0.1) con modificador 1.7 suma 0.17 a posicionX solo si está vivo y no ataca. El motor ya calculó la distancia base y sus límites.

Pregunta para ensayar

¿Por qué getSaludMaxima además de getSalud? Permite calcular porcentajes y mitad de vida sin perder el valor inicial al recibir daño.

Fragmento real de mover() · inicio; los métodos largos se recortan aquí.

```
public void mover(double distancia) {

    if (!atacando && estaVivo()) {

        posicionX += distancia * modificadorVelocidad;
    }
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Zombi() Línea(s) 17	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getSalud() Línea(s) 36	Consulta y devuelve salud. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getSaludMaxima() Línea(s) 40	Consulta y devuelve saludMaxima. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getDanio() Línea(s) 44	Consulta y devuelve danio. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getCosto() Línea(s) 48	Consulta y devuelve costo. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getModificadorVelocidad() Línea(s) 52	Consulta y devuelve modificadorVelocidad. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getFila() Línea(s) 56	Consulta y devuelve fila. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getPosicionX() Línea(s) 60	Consulta y devuelve posicionX. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
estaAtacando() Línea(s) 64	Consulta la bandera que impide mover mientras ejecuta un ataque normal.

Método / ubicación	Explicación
recibirDanio() Línea(s) 68	Resta salud y la limita a cero; las clases con escudo redefinen esta operación.
estaVivo() Línea(s) 77	Devuelve salud > 0 para que motor y dibujo omitan muertos.
perdioBrazo() Línea(s) 81	Compara salud actual con saludMaxima / 2; comunica un estado visual, no reduce daño de ataque por sí mismo.
iniciarAtaque() Línea(s) 85	Pone atacando = true; mover() dejará de avanzar.
detenerAtaque() Línea(s) 89	Pone atacando = false para permitir movimiento cuando corresponda.
mover() Línea(s) 93	Si no ataca y está vivo, suma distancia × modificadorVelocidad a posicionX.
setModificadorVelocidad() Línea(s) 101	Setter protected usado por Garrocha para pasar a velocidad 1.0 después del uso.
setPosicionX() Línea(s) 105	Setter protected que asigna la coordenada; está disponible para hijas y no equivale a la validación de compra.
setFila() Línea(s) 109	Setter protected que asigna fila; las operaciones de Partida validan límites cuando agregan entidades.

25. ZombiComun

Paquete: Zombis · **Archivo:** zombis/ZombiComun.java

Responsabilidad

Configura el atacante básico sin atributos adicionales.

Atributos y relaciones

El constructor envía a super: salud 100, daño 15, costo 50, velocidad 1.0 y la ubicación recibida.

Un recorrido concreto

Una compra lo construye en x = 0 del carril elegido. Su daño, movimiento y pérdida de brazo se obtienen de la clase base y del motor.

Pregunta para ensayar

¿Por qué la clase es pequeña? Reutiliza lo heredado y solo necesita fijar parámetros específicos.

Fragmento real de ZombiComun() · inicio; los métodos largos se recortan aquí.

```
    public ZombiComun(int fila, double posicionX) {

        super(
            100,  // Salud
            15,   // Daño
            50,   // Costo
            1.0,  // Velocidad
            fila,
            posicionX
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
ZombiComun() Línea(s) 6	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.

26. ZombiCaracono

Paquete: Zombis · **Archivo:** zombis/ZombiCaracono.java

Responsabilidad

Agrega escudo y redefine el daño recibido y el daño de ataque al perder el cono.

Atributos y relaciones

100 HP, daño inicial 20, costo 100, velocidad 1.0 y escudoCono = 100.

Un recorrido concreto

Con escudo 30 y golpe 50, calcula restante 20, pone escudo en cero y super.recibirDanio(20). getDanio() pasa a devolver 15 sin cono.

Pregunta para ensayar

¿Por qué no se pierde salud mientras alcanza el escudo? recibirDanio() consume primero ese valor y solo entrega el excedente a la implementación padre.

Fragmento real de recibirDanio() · inicio; los métodos largos se recortan aquí.

```
    public void recibirDanio(int cantidad) {

        if (escudoCono > 0) {

            if (cantidad <= escudoCono) {

                escudoCono -= cantidad;

            } else {
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
ZombiCaracono() Línea(s) 7	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getEscudoCono() Línea(s) 14	Consulta y devuelve escudoCono. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
tieneCono() Línea(s) 18	Devuelve si escudoCono todavía es positivo.
getDanio() Línea(s) 23	Redefine la consulta: devuelve 20 con cono y 15 sin él.
recibirDanio() Línea(s) 26	Consume primero escudoCono; si sobra daño lo pasa a super.recibirDanio. Sin escudo delega todo el golpe al padre.

27. ZombiGarrocha

Paquete: Zombis · **Archivo:** zombis/ZombiGarrocha.java

Responsabilidad

Conserva el uso único de garrocha y el cambio de velocidad.

Atributos y relaciones

120 HP, daño 15, costo 100, velocidad inicial 1.7; tieneGarrocha = true y saltoRealizado = false.

Un recorrido concreto

realizarSalto() cambia las dos banderas y la velocidad a 1.0. El motor solo lo traslada detrás de la planta si no es Nuez; contra Nuez consume la garrocha sin superarla.

Pregunta para ensayar

¿La animación produce el salto lógico? No. El motor actualiza posición y la vista consulta saltoHasta para el efecto visual.

Fragmento real de realizarSalto() · inicio; los métodos largos se recortan aquí.

```
    public void realizarSalto() {

        if (!saltoRealizado) {

            saltoRealizado = true;
            tieneGarrocha = false;

            setModificadorVelocidad(1.0);
        }
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
ZombiGarrocha() Línea(s) 8	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
tieneGarrocha() Línea(s) 16	Consulta si conserva el instrumento; se usa para seleccionar su representación.
haSaltado() Línea(s) 20	Consulta la bandera de uso de garrocha para impedir repeticiones.
realizarSalto() Línea(s) 24	Si todavía no se usó, marca saltoRealizado, elimina tieneGarrocha y reduce la velocidad a 1.0. La ubicación posterior la resuelve el motor.

28. ZombiDeportista

Paquete: Zombis · **Archivo:** zombis/ZombiDeportista.java

Responsabilidad

Agrega casco, velocidad alta y frecuencia de mordisco propia.

Atributos y relaciones

100 HP, daño 10, costo 150, velocidad 2.0; escudoCasco = 150 e intervaloAtaque = 500 ms.

Un recorrido concreto

Golpe de 180 contra casco intacto elimina 150 de escudo y resta 30 de salud. Ataca de nuevo cuando el motor alcanza su próximo intervalo de medio segundo.

Pregunta para ensayar

¿El intervalo está en segundos? Se almacena en milisegundos; el motor divide entre 1000.0 para compararlo con su reloj en segundos.

Fragmento real de recibirDanio() · inicio; los métodos largos se recortan aquí.

```
    public void recibirDanio(int cantidad) {

        if (escudoCasco > 0) {

            if (cantidad <= escudoCasco) {

                escudoCasco -= cantidad;

            } else {
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
ZombiDeportista() Línea(s) 8	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getEscudoCasco() Línea(s) 16	Consulta y devuelve escudoCasco. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getIntervaloAtaque() Línea(s) 20	Consulta y devuelve intervaloAtaque. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
tieneCasco() Línea(s) 24	Consulta si el escudo sigue siendo positivo; la vista lo usa para el estado gráfico.
recibirDanio() Línea(s) 29	Consume primero el casco y aplica solo el excedente a salud mediante super; sin casco aplica todo el golpe a salud.

29. ZombiBailon

Paquete: Zombis · **Archivo:** zombis/ZombiBailon.java

Responsabilidad

Conserva el estado de baile y sus intervalos; el motor crea bailarines.

Atributos y relaciones

150 HP, daño 15, costo 150, velocidad 0.8; intervaloBaile = 5000 ms, duracionBaile = 1000 ms y bailando = false.

Un recorrido concreto

A los 5 s programados inicia baile e intenta invocar adyacentes. Mientras ahora < terminaBaile no avanza; al terminar vuelve al comportamiento normal.

Pregunta para ensayar

¿Siempre aparecen cuatro bailarines? No. Cada ubicación debe estar dentro del jardín y libre de planta viva y zombi vivo.

Fragmento real de ZombiBailon() · inicio; los métodos largos se recortan aquí.

```
public ZombiBailon(int fila, double posicionX) {

    super(150, 15, 150, 0.8, fila, posicionX);

    intervaloBaile = 5000;
    duracionBaile = 1000;
    bailando = false;
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
ZombiBailon() Línea(s) 9	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
getIntervaloBaile() Línea(s) 18	Consulta y devuelve intervaloBaile. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
getDuracionBaile() Línea(s) 22	Consulta y devuelve duracionBaile. Expone este dato a quien coordina, dibuja o guarda el estado; consultar no realiza un ataque ni una compra.
estaBailando() Línea(s) 26	Consulta el estado que el motor utiliza para detener temporalmente el avance.
iniciarBaile() Línea(s) 30	Activa bailando; el motor programa fin y crea los bailarines disponibles.
terminarBaile() Línea(s) 34	Desactiva bailando cuando vence su plazo.

30. ZombiBailarin

Paquete: Zombis · **Archivo:** zombis/ZombiBailarin.java

Responsabilidad

Es una entidad secundaria creada por Bailón, sin compra directa.

Atributos y relaciones

Hereda los datos configurados por super: 100 HP, daño 15, costo 0 y velocidad 1.0.

Un recorrido concreto

invocarBailarines() lo crea y agrega con agregarZombiGenerado(). No se descuenta dinero ni aumenta el contador de compras.

Pregunta para ensayar

¿Costo cero significa comprable gratis? No: la compra rechaza costos no positivos. Esta entidad entra por el método de generación.

Fragmento real de ZombiBailarin() · inicio; los métodos largos se recortan aquí.

```
    public ZombiBailarin(int fila, double posicionX) {

        super(
            100,
            15,
            0,
            1.0,
            fila,
            posicionX
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
ZombiBailarin() Línea(s) 6	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.

31. Zombistein

Paquete: Zombis · **Archivo:** zombis/Zombistein.java

Responsabilidad

Conserva si ya lanzó al Zombidito y permite consultar el umbral.

Atributos y relaciones

300 HP, daño base 0, costo 350, velocidad 0.7 y zombiditoLanzado = false. El daño cero se debe a que aplasta por una regla del motor.

Un recorrido concreto

Al quedar en 150 HP o menos, debeLanzarZombidito() es true si no lanzó. El motor genera al hijo y marca la bandera para evitar repeticiones.

Pregunta para ensayar

¿Por qué guardar la bandera? De lo contrario, mientras siguiera bajo la mitad lanzaría otro hijo en cada actualización.

Fragmento real de debeLanzarZombidito() · inicio; los métodos largos se recortan aquí.

```
public boolean debeLanzarZombidito() {

    return getSalud() <= getSaludMaxima() / 2
            && !zombiditoLanzado;
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Zombistein() Línea(s) 7	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
haLanzadoZombidito() Línea(s) 14	Consulta la bandera; la vista puede mostrar al gigante sin el hijo.
debeLanzarZombidito() Línea(s) 18	Exige salud a la mitad o menos y que no haya lanzado todavía.
marcarZombiditoComoLanzado() Línea(s) 24	Pone la bandera a true después de que el motor crea al hijo, evitando lanzamientos repetidos.

32. Zombidito

Paquete: Zombis · **Archivo:** zombis/Zombidito.java

Responsabilidad

Es la entidad secundaria que llega mediante el lanzamiento del gigante.

Atributos y relaciones

50 HP, daño 15, costo 0, velocidad 1.7. Sus plazos y origen de vuelo están en arreglos del motor.

Un recorrido concreto

El modelo lo ubica en destino y la vista interpola desde el origen durante 0.8 s con arco senoidal. Mientras vuela se excluye de ataques y colisiones normales.

Pregunta para ensayar

¿Se serializa el avance del vuelo? Se conservan los plazos y origen con el motor; la pausa congela su reloj interno.

Fragmento real de Zombidito() · inicio; los métodos largos se recortan aquí.

```
    public Zombidito(int fila, double posicionX) {

        super(
            50,
            15,
            0,
            1.7,
            fila,
            posicionX
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Zombidito() Línea(s) 5	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.

33. Recursos

Paquete: Vista · **Archivo:** vista/Recursos.java

Responsabilidad

Centraliza la búsqueda, conversión y escalado de imágenes.

Atributos y relaciones

No conserva un estado compartido; constructor privado y métodos estáticos de utilidad.

Un recorrido concreto

icono("zombi.png") busca rutas de archivos y luego /Imagenes dentro del classpath. iconoEscalado() usa la menor escala para que quepa sin deformarse.

Pregunta para ensayar

¿Por qué getResource? Puede encontrar un recurso incluido dentro del JAR, donde no existe como archivo ordinario del disco.

Fragmento real de iconoEscalado() · inicio; los métodos largos se recortan aquí.

```
public static ImageIcon iconoEscalado(String nombre, int ancho, int alto) {
    ImageIcon original = icono(nombre);
    if (original.getIconWidth() <= 0) return null;
    double escala = Math.min((double)ancho / original.getIconWidth(), (double)alto /
    original.getIconHeight());
    return new ImageIcon(original.getImage().getScaledInstance(Math.max(1,
    (int)(original.getIconWidth() * escala)),
        Math.max(1, (int)(original.getIconHeight() * escala)), Image.SCALE_SMOOTH));
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
Recursos() Línea(s) 10	Constructor privado para utilizar la clase como utilidad mediante métodos estáticos.
icono() Línea(s) 11	Busca el archivo en tres rutas y después en /Imagenes del classpath. Si no encuentra devuelve ImageIcon vacío, no null.
imagen() Línea(s) 20	Convierte un icono válido a BufferedImage ARGB dibujándolo sobre un Graphics2D que luego libera; devuelve null si el icono no tiene dimensiones.
iconoEscalado() Línea(s) 27	Consulta icono, calcula la menor razón para encajar en el ancho/alto pedido y usa getScaledInstance conservando proporción. Si falta la imagen devuelve null.

34. PanelFondo

Paquete: Vista · **Archivo:** vista/PanelFondo.java

Responsabilidad

Es un JPanel que dibuja la imagen de fondo conservando proporción.

Atributos y relaciones

imagen referencia el fondo; el LayoutManager recibido se configura mediante el constructor padre.

Un recorrido concreto

paintComponent() calcula la mayor razón ancho/alto para cubrir el panel, centra la imagen y superpone negro semitransparente para mejorar lectura de controles.

Pregunta para ensayar

¿Por qué llamar super.paintComponent? Permite la limpieza y pintura básica del componente antes de añadir su representación propia.

Fragmento real de paintComponent() · inicio; los métodos largos se recortan aquí.

```
@Override protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        if (imagen == null) return;
        double escala = Math.max((double)getWidth() / imagen.getWidth(this), (double)getHeight() /
    imagen.getHeight(this));
        int w = (int)Math.ceil(imagen.getWidth(this) * escala), h =
    (int)Math.ceil(imagen.getHeight(this) * escala);
        g.drawImage(imagen, (getWidth() - w) / 2, (getHeight() - h) / 2, w, h, this);
        g.setColor(new Color(0, 0, 0, 65)); g.fillRect(0, 0, getWidth(), getHeight());
    }
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
PanelFondo() Línea(s) 9	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
paintComponent() Línea(s) 13	Llama al padre, calcula escala de cobertura con Math.max, centra y dibuja el fondo y una capa negra con transparencia. No cambia el archivo original.

35. PanelJardinFondo

Paquete: Vista · **Archivo:** vista/PanelJardinFondo.java

Responsabilidad

Dibuja el tablero y entidades y convierte clics a recolección o selección de carril.

Atributos y relaciones

Referencias Jardin/Partida/Motor y callback SeleccionCarril; arreglos e imágenes de entidades; casaDerecha. AreaCesped agrupa límites, escalas y fondo para compartirlos entre dibujo y clic.

Un recorrido concreto

Con borde superior 100, altoCasilla 60 y clic vertical 250, la fila es int((250−100)/60) = 2. Primero se comprueban los iconos de cerebros para no comprar detrás de ellos.

Pregunta para ensayar

¿Pinta y mueve a la vez? No se avanza el combate dentro del pintado. El panel lee el modelo; fotograma() actualiza las reglas antes de solicitar repaint().

Fragmento real de mouseClicked() · inicio; los métodos largos se recortan aquí.

```
            public void mouseClicked(MouseEvent evento) {
                if (seleccionCarril == null || jardin == null || partida == null ||
    partida.estaPausada() || partida.estaTerminada()) return;
                AreaCesped area = calcularAreaCesped();
                if (area == null) return;
                // Recoger primero el icono superior; nunca invocar a traves de un cerebro.
                if (motorCombate != null) {
                    for (int i = motorCombate.getCantidadCerebros() - 1; i >= 0; i--) {
                        var cerebro = motorCombate.getCerebro(i);
                        double x = area.x + cerebro.getX() * area.anchoCasilla;
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
PanelJardinFondo() Línea(s) 67	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
mouseClicked() Línea(s) 107	Rechaza estado no jugable; calcula AreaCesped, prueba primero cerebros desde el último al primero y, si no recoge, valida límites y notifica la fila.
cargarImagen() Línea(s) 135	Pide la carga opcional y devuelve imagen o null; null permite recurrir al dibujo vectorial de respaldo.
cargarImagenOpcional() Línea(s) 143	Usa Recursos.imagen; si falta un .png intenta el mismo nombre base .jpg y .gif antes de devolver null.
setJardin() Línea(s) 154	Asigna la referencia Jardin y solicita repaint; no reconstruye el modelo.
setPartida() Línea(s) 159	Asigna Partida y solicita repaint para representar sus entidades y estado.
setMotorCombate() Línea(s) 164	Asigna el motor para consultar proyectiles, cerebros y efectos y solicita repaint.
setSeleccionCarril() Línea(s) 169	Registra el callback que recibirá el índice de fila cuando se haga clic válido.

Método / ubicación	Explicación
calcularAreaCesped() Línea(s) 182	Determina fondo según dimensiones, escala centrada, límites de césped y tamaño de celda. Refleja los límites cuando la casa está a la derecha; devuelve null si no se puede medir.
paintComponent() Línea(s) 230	Limpia mediante super, crea Graphics2D y pinta fondo, cuadrícula si hace falta, plantas, efectos, proyectiles, zombis, podadoras y al final cerebros. Libera el contexto en finally.
dibujarPodadoras() Línea(s) 381	Consulta disponibilidad o progreso del motor y dibuja una podadora por carril; interpola su barrido desde la derecha hacia la izquierda.
dibujarProyectiles() Línea(s) 414	Convierte posiciones lógicas a píxeles y dibuja círculos verdes o azules según esHielo().
saludMaximaPlanta() Línea(s) 428	Devuelve máximo para la barra de salud: Nuez 400, Patatapum 50, Pinchohierba 1 y otras 100.
dibujarZombis() Línea(s) 436	Omite muertos, determina tamaño y posición visual, vuelo/salto/baile, sprite y estados; dibuja barra de salud y usa respaldo vectorial si falta imagen.
dibujarEstadoPlanta() Línea(s) 505	Representa grietas de Nuez, indicador de Patatapum y texto de digestión sin cambiar el estado lógico.
dibujarCerebros() Línea(s) 521	Dibuja iconos rosados al final, sobre otras entidades; utiliza la misma conversión que el cálculo del clic.
spriteZombi() Línea(s) 532	Elige variante por tipo y estado: bailarín/hijo, gigante sin hijo, brazo, cono/casco o garrocha. Usa imágenes base como respaldo cuando corresponda.
indiceZombi() Línea(s) 553	Relaciona el tipo con un índice 0–5 del arreglo base; bailarín e hijo comparten índice base común si no se usa su variante.
debeMirarIzquierda() Línea(s) 565	Devuelve true para índices de Lanzaguisantes, Hielaguisantes, Carroñívora y Bipetidora, para orientar ataques hacia los zombis.
indicePlanta() Línea(s) 569	Relaciona cada clase de planta con un índice 0–6 de sprites y devuelve −1 para desconocida.
dibujarMarcador() Línea(s) 580	Si no hay sprite, dibuja un círculo con abreviatura de la planta; calcula centrado con FontMetrics.
alSeleccionarCarril() Línea(s) 61	Método de la interfaz interna SeleccionCarril: contrato que recibe una fila. VentanaPartida aporta la implementación mediante una lambda; el panel solo notifica.

36. HilosPartida

Paquete: Vista · **Archivo:** vista/HilosPartida.java

Responsabilidad

Administra un trabajador de reloj y uno por generador; publica tareas en el EDT.

Atributos y relaciones

motor y fotograma son referencias final; reloj y generadores conservan Thread; activo y sesion invalidan trabajos pendientes. El control se exige desde el EDT.

Un recorrido concreto

Pausar pone activo en false y aumenta sesion antes de interrumpir. Una entrega antigua en la cola compara su sesión capturada y retorna sin actuar.

Pregunta para ensayar

¿Por qué no se necesita leer activo desde los trabajadores? El bucle trabajador comprueba su interrupción; activo y sesion se consultan en la tarea del EDT, no en ese bucle.

Fragmento real de trabajador() · inicio; los métodos largos se recortan aquí.

```
    private Thread trabajador(String nombre, int actual, Runnable tarea) {
        // La validacion se ejecuta en el EDT, no en el hilo trabajador.
        Runnable entrega = () -> { if (activo && sesion == actual) tarea.run(); };
        Thread hilo = new Thread(() -> {
            while (!Thread.currentThread().isInterrupted()) {
                try { Thread.sleep(40); }
                catch (InterruptedException e) { Thread.currentThread().interrupt(); return; }
                SwingUtilities.invokeLater(entrega);
            }
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
HilosPartida() Línea(s) 19	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
exigirEDT() Línea(s) 23	Lanza IllegalStateException si se intenta controlar los trabajadores fuera del hilo de eventos de Swing.
iniciar() Línea(s) 26	Evita doble inicio, activa sesión, usa modo externo y crea reloj y un trabajador por generador; cada lambda captura su índice.
trabajador() Línea(s) 39	Construye Thread daemon; espera 40 ms y publica entrega en EDT hasta interrupción. entrega exige sesión activa antes de ejecutar la tarea.
detener() Línea(s) 51	Desde EDT desactiva, cambia sesión e interrumpe/limpia referencias de los trabajadores; desactiva modo externo.
getCantidadGeneradoresActivos() Línea(s) 60	Cuenta referencias a Thread que siguen vivas y exige realizar la consulta en EDT.

37. VentanaInicio

Paquete: Vista · **Archivo:** vista/VentanaInicio.java

Responsabilidad

Primera pantalla con crear/seleccionar cuenta y navegación de teclado.

Atributos y relaciones

gestorCuentas compartido; botones jButton1/2 y etiquetas. initComponents() configura componentes y oyentes.

Un recorrido concreto

Clic en Crear ejecuta jButton1ActionPerformed, construye VentanaCrearCuenta con el mismo gestor, la muestra y llama dispose() sobre la ventana anterior.

Pregunta para ensayar

¿Qué hace Enter? Mediante InputMap/ActionMap provoca doClick() sobre el botón que tiene el foco; reutiliza el evento de ratón.

Fragmento real de jButton1ActionPerformed() · inicio; los métodos largos se recortan aquí.

```
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt)
    {//GEN-FIRST:event_jButton1ActionPerformed
        // TODO add your handling code here:
    VentanaCrearCuenta ventana =
                new VentanaCrearCuenta(gestorCuentas);

        ventana.setLocationRelativeTo(null);
        ventana.setVisible(true);

        this.dispose();
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
VentanaInicio() Línea(s) 9	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
configurarTeclado() Línea(s) 24	Relaciona teclas con acciones mediante InputMap y ActionMap; cambia el foco y confirma con doClick para reutilizar manejadores existentes.
actionPerformed() Línea(s) 41, 56, 71, 111, 120 Sobrecargas o callbacks agrupados	Implementaciones anónimas de ActionListener/AbstractAction. Algunas ejecutan navegación de foco y otras delegan al manejador nombrado; no son clases principales adicionales.
cargarFondo() Línea(s) 82	Busca la imagen de esa pantalla mediante getResource y la asigna si existe.
initComponents() Línea(s) 96	Código de construcción de Swing: crea controles, textos, tamaños/posiciones y registra oyentes. Los bloques GEN-BEGIN/END son del diseñador de NetBeans, no reglas de combate.
jButton1ActionPerformed() Línea(s) 134	Construye VentanaCrearCuenta con el mismo gestor, centra y muestra, y libera la ventana inicial con dispose().
jButton2ActionPerformed() Línea(s) 145	Abre VentanaSeleccionarCuenta con el gestor compartido y cierra esta ventana.

38. VentanaCrearCuenta

Paquete: Vista · **Archivo:** vista/VentanaCrearCuenta.java

Responsabilidad

Recibe el nickname y coordina con el gestor la creación y apertura del menú.

Atributos y relaciones

GestorCuentas, campo jTextField1, botones y etiquetas; acciones de teclado para cambiar foco y confirmar.

Un recorrido concreto

El texto " Ana1 " pasa por trim() y se intenta crear como Ana1. Un espacio interno o símbolo se rechaza; si crea, recupera la Cuenta y abre su menú.

Pregunta para ensayar

¿Por qué configurar teclas también en el campo? JTextField captura flechas; su InputMap/ActionMap permite aplicar las acciones definidas cuando tiene el foco.

Fragmento real de jButton1ActionPerformed() · inicio; los métodos largos se recortan aquí.

```
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt)
    {//GEN-FIRST:event_jButton1ActionPerformed
        // TODO add your handling code here:

   String nickname = jTextField1.getText().trim();

    if (gestorCuentas.crearCuenta(nickname)) {

        javax.swing.JOptionPane.showMessageDialog(
                this,
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
VentanaCrearCuenta() Línea(s) 14	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
configurarTeclado() Línea(s) 30	Relaciona teclas con acciones mediante InputMap y ActionMap; cambia el foco y confirma con doClick para reutilizar manejadores existentes.
actionPerformed() Línea(s) 35, 43, 51, 59, 67, 156, 165, 175 Sobrecargas o callbacks agrupados	Implementaciones anónimas de ActionListener/AbstractAction. Algunas ejecutan navegación de foco y otras delegan al manejador nombrado; no son clases principales adicionales.
configurarTecla() Línea(s) 88	Registra una KeyStroke y Action en raíz y también en JTextField, que intercepta algunas teclas cuando tiene foco.
cargarFondo() Línea(s) 116	Busca la imagen de esa pantalla mediante getResource y la asigna si existe.
initComponents() Línea(s) 131	Código de construcción de Swing: crea controles, textos, tamaños/posiciones y registra oyentes. Los bloques GEN-BEGIN/END son del diseñador de NetBeans, no reglas de combate.
jButton1ActionPerformed() Línea(s) 190	Obtiene y recorta texto con trim; intenta crear cuenta; muestra mensaje y, si tiene éxito, recupera Cuenta, abre menú y cierra formulario.

Método / ubicación	Explicación
jButton2ActionPerformed() Línea(s) 220	Cancela creando de nuevo VentanaInicio con el gestor y cierra el formulario.
jTextField1ActionPerformed() Línea(s) 231	Provoca doClick en el botón de crear para confirmar desde el campo.

39. VentanaSeleccionarCuenta

Paquete: Vista · **Archivo:** vista/VentanaSeleccionarCuenta.java

Responsabilidad

Presenta las cuentas existentes y navega al menú de la seleccionada.

Atributos y relaciones

GestorCuentas compartido, JComboBox de nicknames, botones y fondo.

Un recorrido concreto

cargarCuentas() limpia el combo y agrega nombres solo hasta cantidadCuentas. Confirmar recupera String seleccionado, busca Cuenta y abre el menú; si no hay selección informa el problema.

Pregunta para ensayar

¿Elegir en el combo inicia una partida? No; la selección conserva el nombre y Continuar abre el menú. El manejador cmbCuentasActionPerformed no añade acciones.

Fragmento real de jButton1ActionPerformed() · inicio; los métodos largos se recortan aquí.

```
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt)
    {//GEN-FIRST:event_jButton1ActionPerformed
        // TODO add your handling code here:
        String nicknameSeleccionado =
            (String) cmbCuentas.getSelectedItem();

    if (nicknameSeleccionado != null) {

        Cuenta cuentaSeleccionada =
                gestorCuentas.buscarCuenta(nicknameSeleccionado);
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
VentanaSeleccionarCuenta() Línea(s) 10	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
configurarTeclado() Línea(s) 28	Relaciona teclas con acciones mediante InputMap y ActionMap; cambia el foco y confirma con doClick para reutilizar manejadores existentes.
actionPerformed() Línea(s) 47, 69, 94, 115, 137, 188, 197, 211 Sobrecargas o callbacks agrupados	Implementaciones anónimas de ActionListener/AbstractAction. Algunas ejecutan navegación de foco y otras delegan al manejador nombrado; no son clases principales adicionales.
cargarFondo() Línea(s) 153	Busca la imagen de esa pantalla mediante getResource y la asigna si existe.
cargarCuentas() Línea(s) 162	Limpia JComboBox, obtiene arreglo y contador y agrega un nickname por cada cuenta utilizada.
initComponents() Línea(s) 175	Código de construcción de Swing: crea controles, textos, tamaños/posiciones y registra oyentes. Los bloques GEN-BEGIN/END son del diseñador de NetBeans, no reglas de combate.
cmbCuentasActionPerformed() Línea(s) 226	Manejador generado sin lógica adicional; seleccionar un elemento no abre por sí solo otra ventana.

Método / ubicación	Explicación
jButton2ActionPerformed() Línea(s) 231	Abre VentanaInicio con el mismo gestor y cierra la selección.
jButton1ActionPerformed() Línea(s) 242	Lee el String del combo, recupera Cuenta y abre su menú; si no hay selección presenta un mensaje.

40. VentanaMenuPrincipal

Paquete: Vista · **Archivo:** vista/VentanaMenuPrincipal.java

Responsabilidad

Coordina nueva partida, recuperación, estadísticas, rankings y salida.

Atributos y relaciones

Referencias final al gestor y a la cuenta activa. Los botones se construyen en el constructor con GridBagLayout y tamaños compactos.

Un recorrido concreto

Reanudar enumera .bin, presenta JList y recupera Estado. Luego crea VentanaPartida con motor, fracción y archivo de origen y cierra el menú.

Pregunta para ensayar

¿El ranking modifica las cuentas originales? Ordena referencias en un arreglo nuevo. getCuentas sí devuelve el arreglo interno; ese getter no entrega una copia defensiva.

Fragmento real de comparar() · inicio; los métodos largos se recortan aquí.

```
private int comparar(Cuenta a, Cuenta b, int tipo) {
    int c = tipo == 0 ? Integer.compare(valor(a, tipo), valor(b, tipo)) : Integer.compare(valor(b,
    tipo), valor(a, tipo));
    return c == 0 ? a.getNickname().compareTo(b.getNickname()) : c;
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
VentanaMenuPrincipal() Línea(s) 12	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
reanudar() Línea(s) 44	Lista binarios de la cuenta, permite elegir uno en JList, carga Estado y construye VentanaPartida pausada con su motor/fracción. Presenta errores o ausencia de archivos.
estadisticas() Línea(s) 57	Construye resumen de cuenta y tres rankings, los coloca en pestañas no editables y muestra un diálogo.
texto() Línea(s) 69	Crea JTextArea no editable con márgenes dentro de JScrollPane para mostrar los textos.
ranking() Línea(s) 70	Recarga cuentas, usa la cuenta activa actualizada, filtra sin victorias para rapidez, ordena por inserción y produce hasta cinco líneas.
valor() Línea(s) 88	Selecciona qué métrica comparar según tipo: tiempo, plantas destruidas o victorias.
comparar() Línea(s) 89	Ordena tiempo ascendente y cantidades descendentes; resuelve empates mediante comparación de nicknames.

41. VentanaNuevaPartida

Paquete: Vista · **Archivo:** vista/VentanaNuevaPartida.java

Responsabilidad

Recibe archivo y dificultad y construye toda la estructura de la partida nueva.

Atributos y relaciones

Gestor y Cuenta final; archivo elegido; etiqueta del nombre y JComboBox con tres dificultades.

Un recorrido concreto

iniciar() exige archivo, lee ConfiguracionNivel, construye Jardin/Partida/Motor y abre VentanaPartida. Un fallo de lectura o validación se convierte en un mensaje de nivel inválido.

Pregunta para ensayar

¿El filtro .json valida su contenido? No: solo ayuda a seleccionar. La validación efectiva la realiza LectorNivel al iniciar.

Fragmento real de iniciar() · inicio; los métodos largos se recortan aquí.

```
private void iniciar() {
    if (archivo == null) { JOptionPane.showMessageDialog(this, "Selecciona un nivel."); return; }
    try {
        ConfiguracionNivel c = new LectorNivel().leerNivel(archivo);
        VentanaPartida v = new VentanaPartida(gestor, cuenta, new MotorCombate(
            new Partida(cuenta, new Jardin(c.getJardin()), c, (String)dificultad.getSelectedItem())),
    0, null);
        v.setVisible(true); dispose();
    } catch (Exception ex) { JOptionPane.showMessageDialog(this, "Nivel inválido: " +
    ex.getMessage()); }
}
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
VentanaNuevaPartida() Línea(s) 16	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super.
seleccionar() Línea(s) 44	Abre JFileChooser desde Niveles con filtro .json; si se acepta conserva File, nombre visible y ruta en tooltip.
iniciar() Línea(s) 52	Exige archivo, lee nivel y construye Jardin, Partida, MotorCombate y VentanaPartida; muestra el juego y cierra configuración. Presenta errores mediante JOptionPane.
volver() Línea(s) 61	Reconstruye el menú de la misma cuenta y libera la ventana actual.
windowClosing() Línea(s) 39	Callback de WindowAdapter declarado dentro del constructor: al pulsar la X llama volver() para mantener la navegación al menú.

42. VentanaPartida

Paquete: Vista · **Archivo:** vista/VentanaPartida.java

Responsabilidad

Une controles, modelo, motor, tablero, trabajadores, pausa, guardado y resultado.

Atributos y relaciones

Gestor, Cuenta, Partida y Motor; tablero; etiquetas; catálogo/costos; seleccionado; HilosPartida; instante/fraccion; archivoOrigen; diálogo y resultadoRegistrado.

Un recorrido concreto

fotograma() mide dt, actualiza reloj y motor y solicita dibujo. Abrir pausa detiene trabajadores; guardar crea un binario; finalizar suma estadísticas y vuelve al menú.

Pregunta para ensayar

¿Para qué archivoOrigen? Al volver a guardar o terminar una partida recuperada permite retirar la copia anterior después de confirmar el nuevo estado o resultado.

Fragmento real de fotograma() · inicio; los métodos largos se recortan aquí.

```
    private void fotograma() {
        if (!isDisplayable() || partida.estaTerminada()) return;
        long ahora = System.nanoTime(); double dt = Math.max(0, (ahora - instante) /
    1_000_000_000.0); instante = ahora;
        if (partida.estaPausada()) return;
        // Limitar TODOS los relojes por igual evita que la simulacion y el contador diverjan.
        dt = Math.min(dt, 0.15);
        fraccion += dt;
        while (fraccion >= 1 && partida.getTiempoRestante() > 0) { fraccion--;
    partida.descontarUnSegundo(); }
        partida.setFraccionReloj(fraccion);
// ... continúa en el archivo fuente
```

Métodos declarados: qué hace cada uno

Método / ubicación	Explicación
VentanaPartida() Línea(s) 30, 33 Sobrecargas o callbacks agrupados	Constructor: inicializa el estado de esta ficha y las referencias recibidas; cuando hereda, ejecuta el constructor padre mediante super. Hay 2 sobrecargas, para distintas formas de inicialización.
windowOpened() Línea(s) 67	Al abrir, muestra diálogo si el modelo ya está pausado; en otro caso inicia trabajadores.
windowClosing() Línea(s) 70	Solicita pausa en lugar de perder directamente el estado al pulsar la X.
windowClosed() Línea(s) 71	Detiene trabajadores y libera el diálogo asociado al cerrar la ventana.
iniciarHilo() Línea(s) 75	Reinicia instante con nanoTime para no contar como juego el tiempo pasado en pausa e inicia HilosPartida.
detenerHilo() Línea(s) 76	Delega detener() en el coordinador de trabajadores.
fotograma() Línea(s) 77	Mide dt, acumula fracción, descuenta segundos, detecta derrota, actualiza motor/etiquetas/dibujo y comprueba victoria.

Método / ubicación	Explicación
actualizarPanel() Línea(s) 90	Actualiza saldo, mm:ss, bloque/horda, precios y estado habilitado del catálogo; indica selección y cursor. Emite aviso sonoro al entrar en hordas.
crear() Línea(s) 104	Switch del tipo elegido a constructor concreto con fila recibida y posicionX = 0; solo incluye seis tipos comprables.
invocar() Línea(s) 112	Exige estado activo y tipo seleccionado; crea y solicita compra al modelo. Si falta saldo pausa mientras informa y reanuda después; si compra limpia selección.
abrirPausa() Línea(s) 124	Marca pausa, detiene trabajadores, actualiza controles y muestra el diálogo sin permitir compras ni recolección.
guardarYSalir() Línea(s) 128	Serializa motor y fracción, retira archivo de origen solo tras confirmar nuevo guardado, detiene/libera ventanas y abre menú. Un error mantiene la posibilidad de reintentar.
finalizar() Línea(s) 137	Guardia resultadoRegistrado evita duplicados. Marca fin, detiene, suma métricas, guarda estadísticas, retira guardado completado y presenta resumen antes de abrir menú.

Recorridos completos para demostrar

Situación	Cómo explicarla y qué comprobar
Cuenta válida/duplicada	VentanaCrearCuenta obtiene texto → trim → crearCuenta → nicknameValido → buscarCuenta → crearArchivosCuenta → Cuenta. Repetir el mismo nickname no crea otro objeto registrado. Ana1 y ana1 sí son distintos.
Nivel irregular	El explorador solo elige File. LectorNivel reconoce matriz y compara anchos; lanza una excepción. VentanaNuevaPartida la captura e informa. No se construye el juego con una matriz irregular.
Compra con descuento	Clic catálogo → índice → clic césped → fila → callback → invocar → crear → Partida.invocarZombi → costoActual. El saldo, gasto y compra cambian únicamente si valida.
Escudo e impacto	dispararPlantas crea Proyectil → avanzar → tramo de colisión → primer blanco → recibirDanio polimórfico → escudo y posible excedente → quitarProyectil → repaint.
Planta destruida	Mordisco o efecto reduce salud → destruirPlanta retira referencia y suma métrica → crea 3–6 Cerebro → clic recoge 25 → final suma métricas y escribe texto.
Pausar con hielo	Pausa detiene trabajadores y bloquea modelo. ahora no avanza: congeladoHasta conserva duración restante. Reanudar reinicia instante de medición para no sumar el tiempo de pausa.
Guardar y continuar	Pausa → Estado(motor,fracción) → ObjectOutputStream temporal → .bin → menú → listar/cargar → filtro/validación → vincularCuenta → pausa → nueva ventana → reanudar nuevos trabajadores.
Resultado una vez	Victoria del motor o cero en reloj → finalizar con guardia → terminar/detener → registrarVictoria/Derrota → sumar contadores → guardarEstadisticasComprobadas → resumen y menú.

Para cada recorrido señala las clases en NetBeans y sigue los valores. Un dato visible en una etiqueta es una representación de un atributo, no el origen de la regla.

Ejercicios con valores y respuestas

Ejercicio	Respuesta y razonamiento
Escudo 30; salud 100; golpe 50	Queda escudo 0 y salud 80. Se consumen 30 del escudo y solo 20 llegan a super.recibirDanio.
150 cerebros; compra 100; luego 150	Primera compra: saldo 50, gasto 100 y una compra. La segunda falla y mantiene esos valores.
Común en bloque 4 y bloque 7	Precio 25 y 13. (50+1)/2 = 25; (50+3)/4 = 13 con división entera.
fraccion 0.97; dt 0.06	La suma da 1.03. Se descuenta un segundo y queda 0.03. Si hubiese varios segundos acumulados, el while puede descontar más de uno.
Deportista, dt 0.1, sin obstáculos	Distancia base 0.44×0.1 = 0.044; Zombi.mover aplica ×2: 0.088 columnas. Con hielo ×0.60: 0.0528.
Hielo impacta a 12 y otra vez a 14	El vencimiento pasa de 16 a 18. La velocidad queda en 60%, sin apilar dos reducciones.
Nuez con salud 264, 260 y 136	264/400 = 66%: estado 1; 260 = 65%: estado 2; 136 = 34%: estado 3. El cálculo es entero.
Patatapum sembrada en segundo 30	preparadaEn = 40. No nace activa por haber transcurrido ya los primeros diez segundos de la partida.
Clic y fila	Si el borde superior es 100, la celda mide 60 y el clic está en y=250: int((250−100)/60) = 2. El primer carril se indexa 0.
Ranking tiempos 40,25,35	Por inserción: se mueve 25 antes de 40 y después 35 antes de 40. Resultado 25,35,40.
Tres victorias de cuatro partidas	75%; con 21 plantas en cuatro partidas, promedio 5.25. Se usa double para conservar decimales.
Primera y segunda llegada al mismo carril	La primera consume podadora y elimina todos los zombis de esa fila; la posterior por el carril ya sin podadora gana.

Decisiones reales y límites que debes conocer

• **JSON específico:** no acepta toda variante del estándar. Exige esquema, orden y conjunto de cadenas. No digas que cualquier JSON válido sirve.

• **Alfanumérico:** Character.isLetterOrDigit admite letras y dígitos de Unicode; no es una comprobación limitada a A–Z y 0–9. El formulario recorta espacios de los extremos con trim.

• **Constantes repetidas:** el motor expresa directamente el hielo como 4 s y factor 0.60, y la preparación de mina como 10 s. Cambiar solo todos los getters de una entidad no garantiza modificar esas reglas.

• **Cadencia de Bipetidora:** se conserva parámetro frontal y trasero, ambos iguales, pero el motor usa el frontal como intervalo común.

• **Cadencias iniciales:** el constructor del motor inicia disparos a 0.3 s; luego programa la cadencia del tipo. No afirmes que el primer disparo debe esperar necesariamente un segundo completo.

• **Garrocha contra Nuez:** el código consume la garrocha y reduce velocidad, pero no permite atravesarla.

• **Muertos:** no se dibujan ni atacan, pero las referencias de zombis muertos permanecen en el arreglo de Partida. Los proyectiles y cerebros retirados sí se compactan.

• **Tiempo de juego:** un dt mayor de 0.15 se limita. Evita saltos grandes, pero una demora prolongada no se recupera totalmente como tiempo de pared.

• **Encapsulamiento:** hay atributos privados, aunque algunos getters devuelven arreglos internos y no copias defensivas. Identifica esa diferencia si te preguntan por mejoras.

• **Persistencia:** serialVersionUID y Estado.version cumplen funciones distintas. La primera es de serialización Java; la segunda es una comprobación del formato de la aplicación.

• **Trabajo concurrente:** hay Threads reales de espera/publicación; el modelo mutable cambia en EDT. invokeLater usa la cola de Swing, no significa que cada callback corra en un hilo nuevo.

• **Dependencias:** Swing, java.io, java.nio y java.util.Random son JDK; usar Random no equivale a usar Collections Framework. Herramientas de documentación y tests están fuera del flujo normal del juego.

Rúbrica: evidencia para cada apartado

Área / puntos	Qué demostrar y explicar
Documentación · 10	Técnico 7: UML, POO/herencia/interfaces, flujo de motor, IA, hilos y archivos; usuario 3: PDF con capturas de todo el flujo. Enseña los dos manuales y su relación con el código.
Inicio y cuentas · 8	Swing 2; creación 3: unicidad, mayúsculas, alfanuméricos 1–12 y carpeta/archivos; selección 1: lista y carga; menú/salida 2: cuatro opciones y navegación.
Configuración/JSON · 8	Selección 1: explorador y dificultad; lectura 4: parser propio, claves, rectángulo, identificadores y errores; jardín 3: dimensiones dinámicas, plantas, entrada, casa y podadoras.
Plantas · 14	Dos puntos por planta: conocer vida, daño, cadencia, preparación/digestión, ralentización, bloqueos y representación visual; mostrar casos normales y excepciones.
Zombis · 14	Dos puntos por tipo evaluado: vida/escudo, costo y velocidad; escudo primero, salto/Nuez, baile y adyacentes, aplastamiento, umbral y vuelo de Zombidito. Distingue Bailón de Bailarín.
Motor/recursos · 15	Panel/reloj 2; generadores/cerebros 4; invocación y error 2; movimiento/impactos/ataques 4; casilla lógica y reglas espaciales 3. Usa recorridos y valores de esta guía.
Dificultad/IA · 8	Fácil 1 sin siembras; Medio 2 cada 10 s desde casa; Difícil 5 cada 5 s con prioridades y una acción. Explica orden de bucles y return.
Hordas/final · 7	Bloques y precios 3; podadora por carril y un uso 2; victoria/derrota, parada y resumen 2. Muestra restauración de precio fuera del bloque 4.
Pausa/guardado · 6	Pausa y continuidad 2; binario con fecha/hora por cuenta 2; listado y recuperación completa en pausa 2. Incluye efectos, salud, posiciones y fracción del reloj.
Estadísticas · 6	Archivo único con ocho datos 3; porcentaje/promedio 1; tres Top 5 con orden correcto 2. Explica inserción y empate por nickname.
Técnica/POO · 4	Arquitectura 2: responsabilidades, encapsulación, herencia/interfaces y main pequeño; hilos/restricciones 2: Threads/EDT, arreglos, JDK, sin Lombok ni globales o sincronización manual prohibida.

Estos son los pesos de la hoja, no una promesa de nota. La evaluación exige que puedas ejecutar y explicar el código que efectivamente tienes en tu equipo.

Fundamentos de Java presentes en tu código

Elemento	Qué significa aquí
Clase, objeto y referencia	La clase define estructura y operaciones. new construye un objeto. Una variable como cuenta conserva una referencia al objeto, no una copia de todos sus datos.
Constructor y this(...)	El constructor inicializa. Cuenta tiene dos formas de construir; Estadisticas() delega con this(0,...,0) a otro constructor. No tiene tipo de retorno.
this y super	this identifica el objeto actual; this.cuenta = cuenta distingue atributo y parámetro. super(...) inicializa la parte heredada; super.recibirDanio(...) llama al comportamiento padre.
private / public / protected	private restringe al ámbito de su clase; public expone una operación; protected permite acceso desde el paquete y subclases según las reglas de Java. En Zombi protege ajustes de posición/velocidad para hijas.
static y final	static pertenece a la clase: RutasCuentas.carpeta(). final impide reasignar esa variable; no hace inmutable todo el objeto referenciado. private static final serialVersionUID es una constante de serialización.
abstract / extends / implements	abstract impide instanciar directamente Planta/Zombi. extends reutiliza una clase padre. implements declara un contrato, como Serializable; SeleccionCarril es una interfaz propia.
@Override y sobrecarga	Override redefine comportamiento heredado, como escudos. Sobrecarga son métodos/constructores del mismo nombre con parámetros distintos, como los constructores de Cuenta.
==, equals y null	== compara referencias si son objetos; indiceZombi busca el mismo objeto. equals compara texto de nicknames. null significa ausencia de referencia, como una casilla sin planta.
&& y ||	Combinan condiciones con cortocircuito. En zombi == null || ... no se consulta zombi.getCosto() si es null. La compra debe pasar todas las reglas antes de modificar saldo.
for, while, do/while	for recorre arreglos/matrices; while repite según condición; do/while analiza al menos un elemento de fila. return termina el método; continue pasa a la siguiente vuelta.
int, double y cast	int sirve para contadores/índices; double para posición y tiempo fraccionario. (int) trunca; Math.floor calcula entero inferior; usar 1000.0 conserva fracciones al convertir milisegundos.
lambda, callback y evento	La lambda describe una tarea; el callback es una operación que otro componente invoca después. Un clic ejecuta su oyente; no recorre todos los archivos abiertos en las pestañas.

Archivos, herramientas y diagnóstico

Elemento	Qué hace y cómo explicarlo
try / catch / finally	try ejecuta una operación que puede fallar; catch trata el tipo de error; finally limpia recursos temporales. No debes confundir capturar un fallo con asegurar que se completó la operación.
try con recursos	Reader, Writer y Object streams se cierran al terminar el bloque incluso si falla. Evita dejar el archivo abierto.
throw y throws	throw lanza una excepción concreta. throws declara que una operación puede propagarla; la ventana puede mostrarla al usuario.
.txt, .json y .bin	JSON define el nivel; TXT conserva estadísticas legibles; BIN conserva el grafo serializado de la partida. Cambiar la extensión no transforma los datos.
Directorio de trabajo	Las rutas relativas cuentas y Niveles dependen de la carpeta desde la que arranca Java. Cambiar de carpeta puede hacer que aparezca otra raíz de cuentas.
JAR y recursos	dist/ZombisVsPlantas.jar contiene clases compiladas y recursos; getResource los busca en classpath. Editar .java no cambia un JAR anterior hasta volver a construirlo.
Pruebas.java	Comprobaciones de lógica, validaciones, persistencia y reglas. Sus mensajes OK son de pruebas, no del arranque normal del juego.
PruebasHilos.java	Comprueba trabajadores, generaciones, pausa/reanudación y que no se dupliquen actividades.
PruebasVisuales.java	Verifica selección de imágenes, estados y vuelo/lanzamiento. Las imágenes y los datos lógicos se comprueban por separado.
PruebasInterfaz y PruebasPartidaCompleta	Ejercitan eventos reales de la ventana, compra, pausa, guardado y resultado por interacción. No son las clases principales del producto.
CapturarManuales.java	Herramienta para capturas del flujo de la aplicación; usa cuentas de demostración. No forma parte del arranque normal.
generar_manuales.py / generar_word.py	Herramientas externas al juego para producir los documentos. Sus bibliotecas no son dependencias para ejecutar el programa Java.

Si algo falla en vivo: identifica la acción exacta, lee el mensaje de la ventana o stack trace de Output, localiza la clase y línea y comprueba entrada/estado. Verifica ruta de nivel, cuenta activa, pausa, saldo, recursos y versión compilada antes de atribuirlo al motor.